

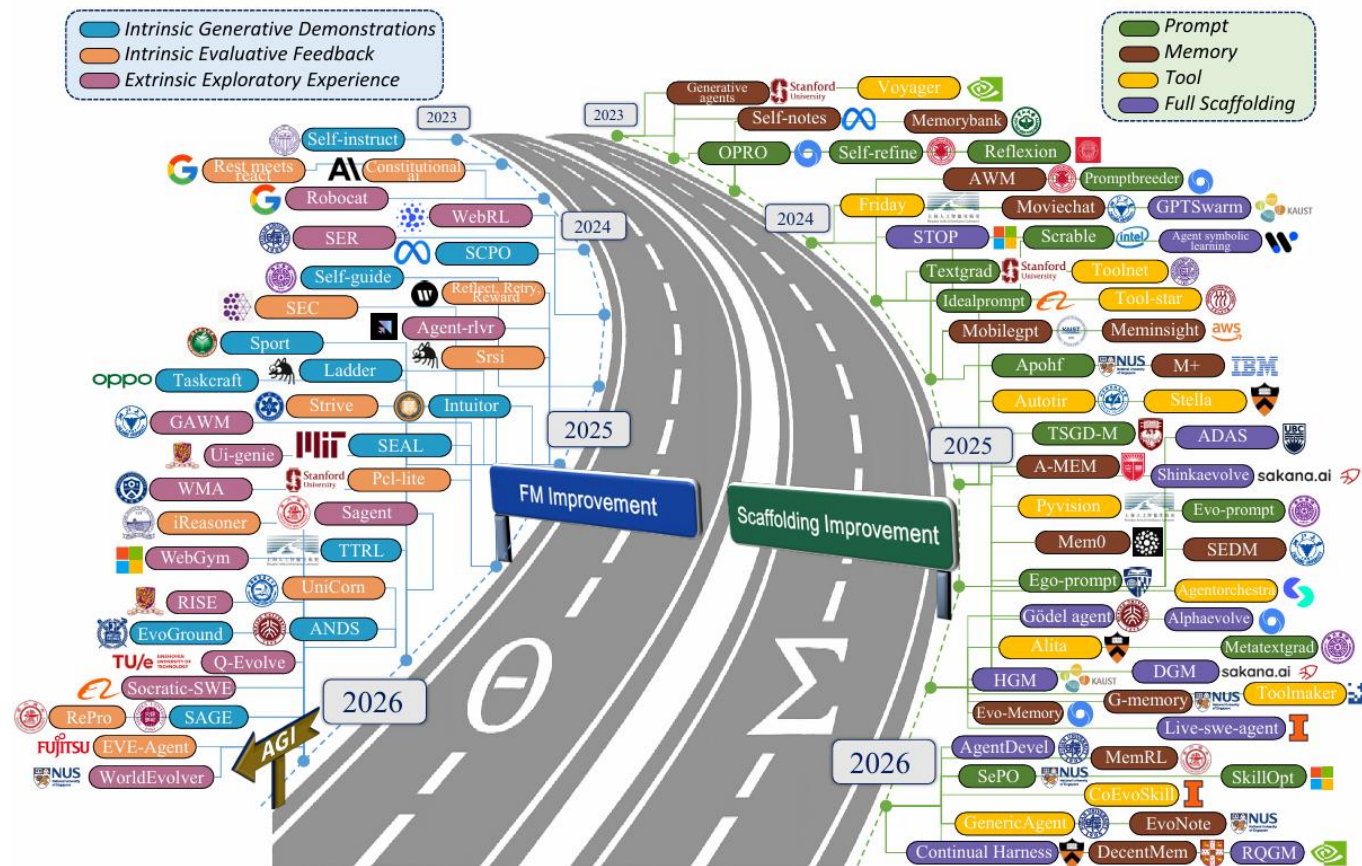
自进化智能体

——研究与实践

肖仰华

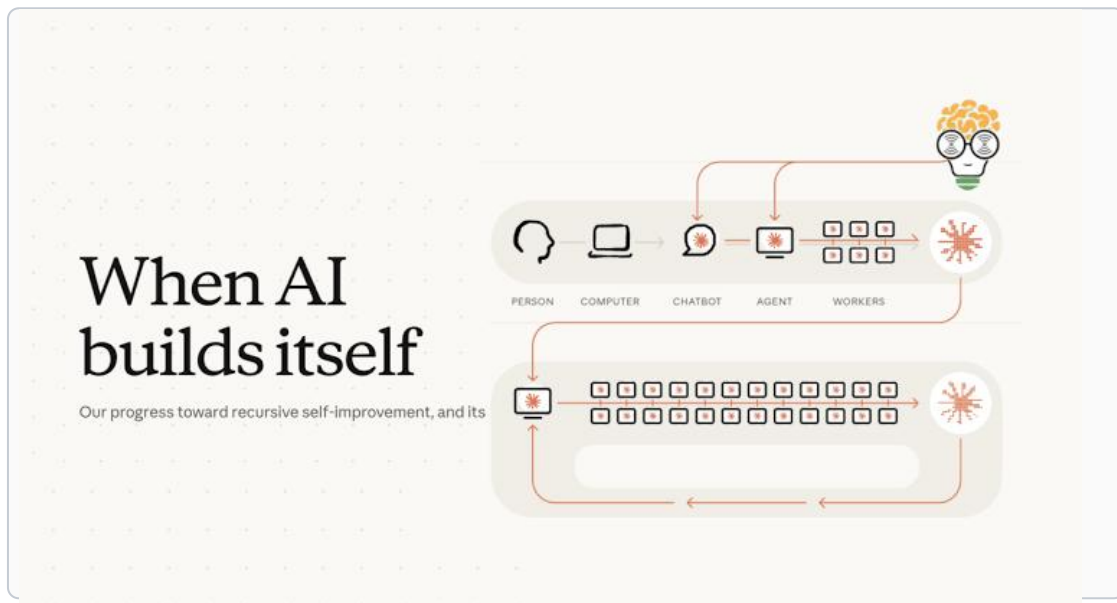
上海市数据科学重点实验室

复旦未来技术研究院

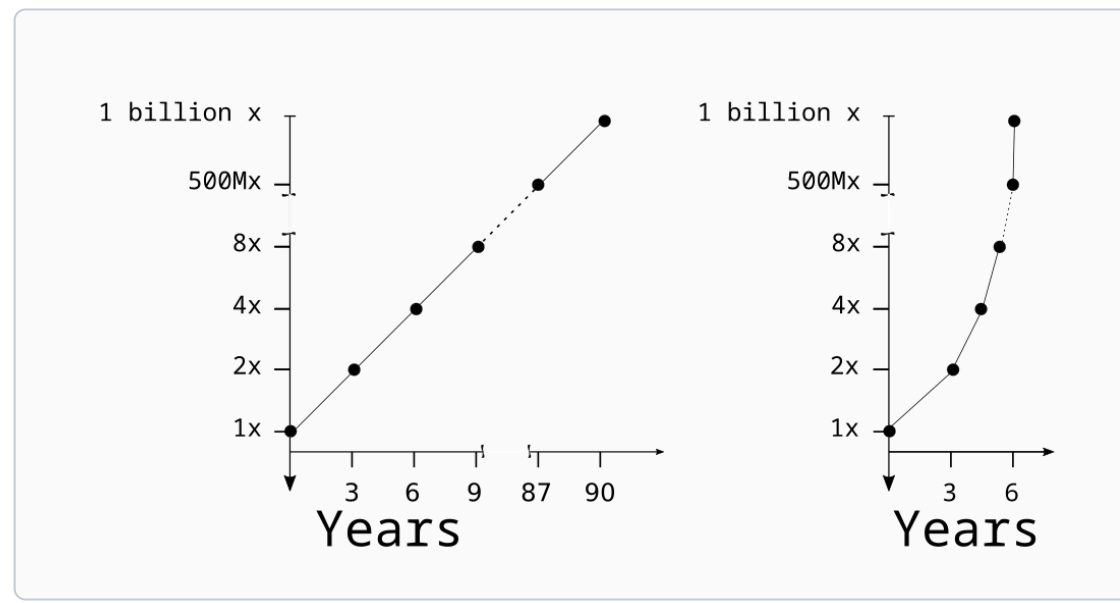


人工智能已经进入自进化时代

- 当AI可以构建自己、改进自己，人工智能已经进入自进化时代；人类社会因此而迎来奇点时刻



典型工程师每季度合并的代码量
相较 2024 年约提高 8 倍



人类社会迈过奇点时刻

自进化智能体技术前沿



自进化智能体引领AI自进化

- 具备自进化能力的智能体已经出现，复杂环境**适应性**、无人干预的**自主性**显著提升

工具自创造

记忆自组织

策略自学习

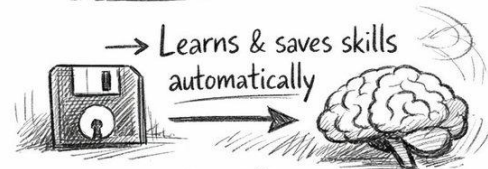
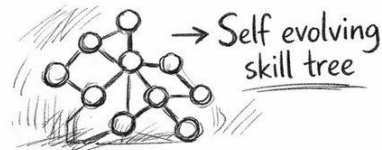
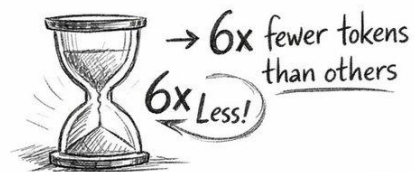
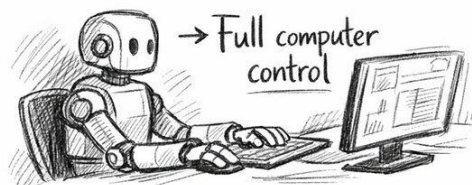
自我审视

自我完善

自进化智能体的主要能力特性

What if your **AI** agent got smarter
every time you used it?

— GenericAgent does exactly that. —



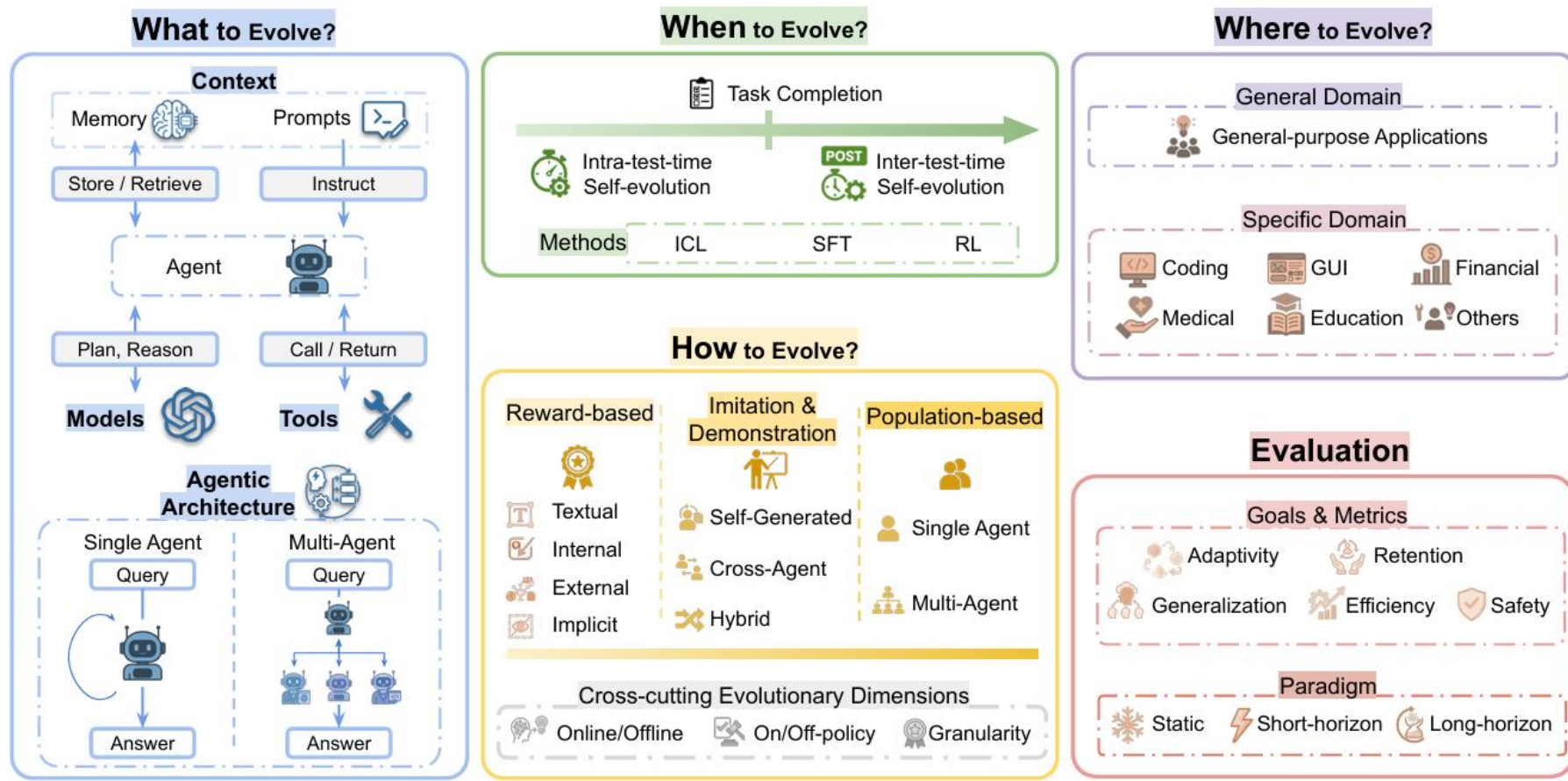
— It literally improves itself. —

100% Open Source. 

与用户共同成长的自进化智能体GenericAgent

几个核心问题

- 自进化Agent不是一个孤立算法, What? When? How? Where? Evaluation?



Agent 自进化：四个闭环

- Agent自进化的关键，是经验沉淀并转化为未来能力。

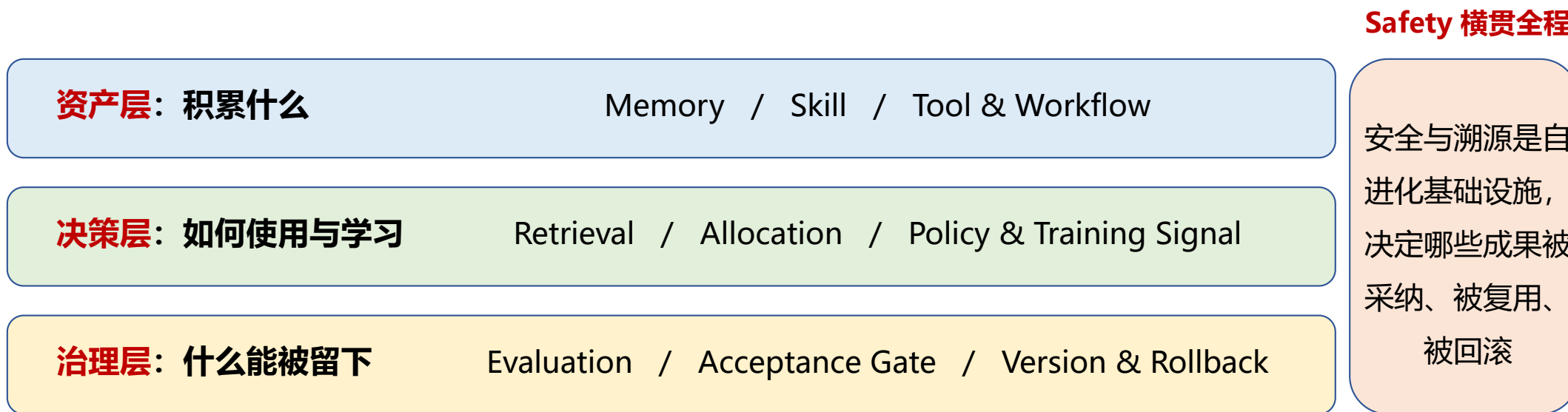
Experience → Update → Commit → Future Capability



自进化的关键不是“有没有反思”，而是经验是否真的经过验证并写回，并转化为未来能力。如果没有 commit 和 continuous evaluation，很多所谓自进化其实只是当前上下文里的临时调整

“资产—决策—治理”视图

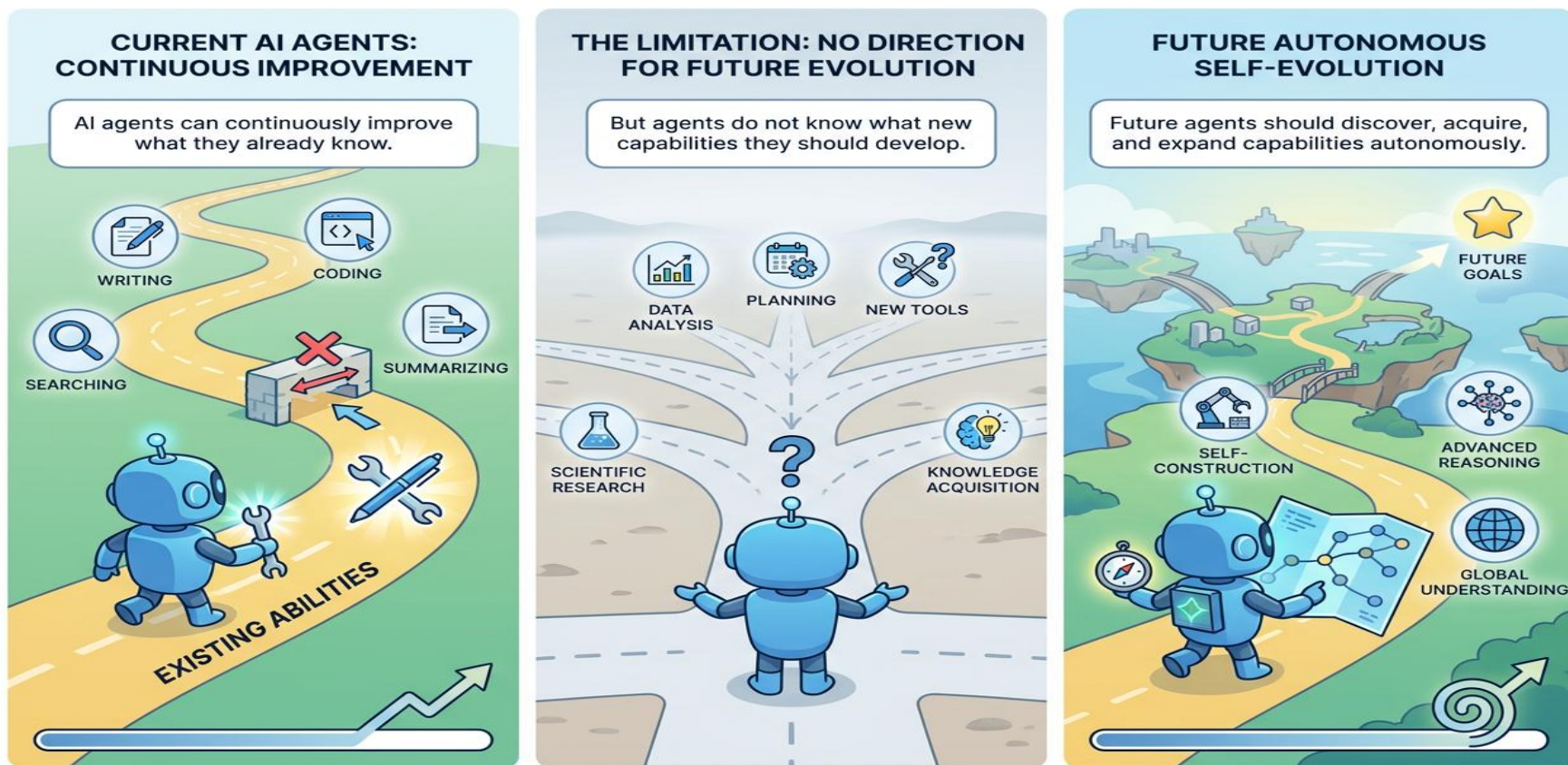
- 自进化要成立，不仅要有会学习的 Agent，还要有**资产沉淀机制**、**决策使用机制**和**治理拦截机制**



Safety / lineage control 贯穿生成、验证、提交和复用，而不是最后再补一道防线。

自主进化方向

- Agent 能持续改进行为，但仍无法自主决定能力应如何扩展，以及未来该向何处进化。

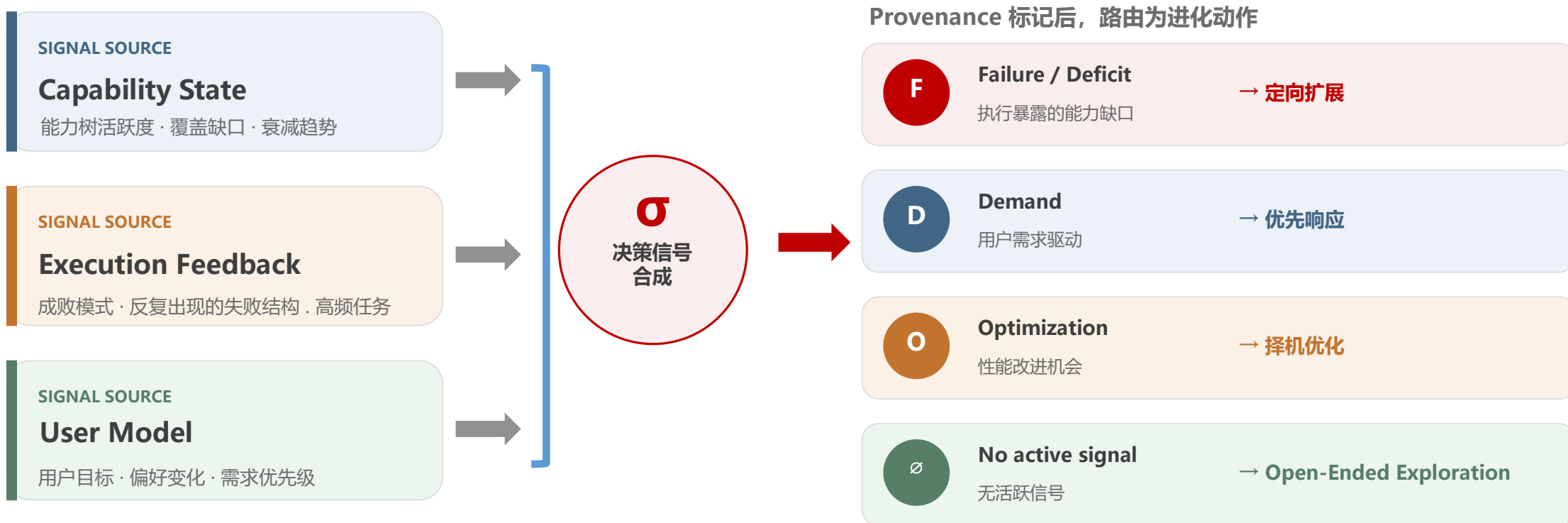


现有方法主要关注在**既定任务目标**下优化已有能力，而未来的 Agent 需要进一步具备**能力发现**与**自主扩展**能力，在长期运行过程中主动决定自身能力发展的方向。

自主进化方向

信号合成与路由：Agent 凭什么决定**进化方向**？

- **自主进化的前提**：从多源信号感知“该往哪走”，并路由为不同的进化动作。



自进化方向由智能体对**能力状态**、**执行反馈**和**用户模型**的综合信号感知决定，有信号时**定向扩展**，无信号时**好奇探索**，实现持续自主进化。

Agent 自进化纵览

方向	代表工作	关键机制	进化价值
Memory	MemSkill / EvolveMem / SelfMem	操作 / 架构 / 策略	持久经验 / 复用
Skill	Trace2Skill / MUSE-Autoskill SkillOS / SkillAxe	轨迹抽取 / 生命周期 库治理 / 诊断修订	技能资产 / 复用 / 更新
Workflow / Policy	AgentEvolver / AFlow / ADAS	经验反馈 / MCTS 搜索 Meta Agent Search	行为 / 结构 / 写回
Training	TT-SI / Q-Evolve / RePro SWE-RL / SAGE	self-training / process rewards self-play tasks / curriculum	参数 / checkpoint 训练课程
Environment	Agent-World / ProPlay Red Queen Gödel Machine	任务环境 / 世界模型 评估标尺共进化	环境 / 标尺 持续压力
User Model	O-Mem / RecNet / GazeMind	active profile / pref. propagation cognitive-load adaptation	用户模型 / 策略适配 隐私与监控
Safety	Safety in SE LLM Agents On Safety Risks	lineage 风险 / 拒答边界 版本验证 / 回滚	治理 / 监控 / 约束
tool	MetaForge / Tool-Star / ToolRL	工具检索 / 适配生成 调用反馈 / 工具回收	工具资产 / 调用策略 / 写回
Architecture	SkillGraph / GEA / Dilemma Game	角色分配 / 拓扑生成 通信更新 / 结构自修复	协作图 / 通信路径 / 群体结构

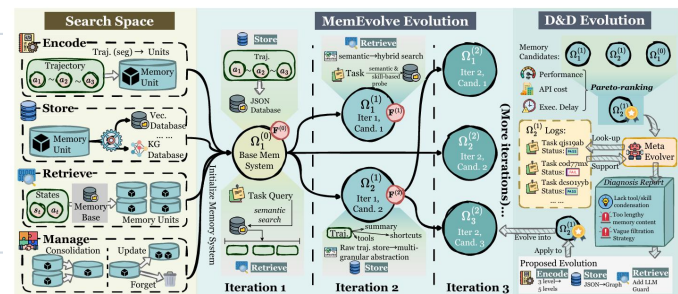
自进化：资产、行为、训练环境、用户状态与安全治理共同进入可验证的长期闭环

Memory进化

- 不只是积累经验，而是让**组织架构**、**操作方式**与**记忆策略**都能随反馈更新。

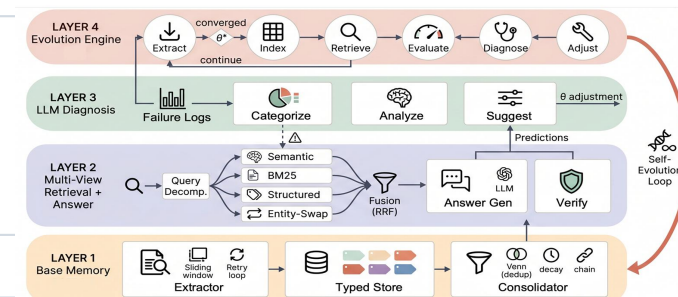
MemEvolve: 操作可进化

让 Agent 的记忆系统**通过进化搜索**（如变异、选择）自动优化记忆的结构与检索策略，使其在持续交互中自适应地改进长期记忆的质量与利用效率。



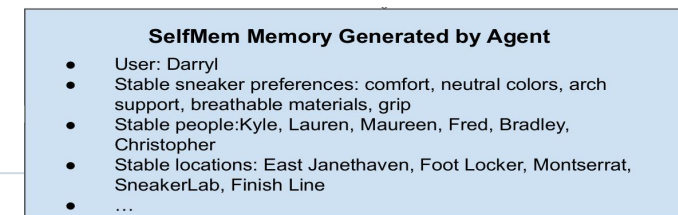
EvolveMem: 架构可演化

记忆系统应同时进化“记了什么”和“如何检索”。EvolveMem通过将检索配置建模为**可优化动作空间**，LLM 根据失败日志自动调整并回归验证，实现记忆架构的闭环自进化与跨 benchmark 迁移。



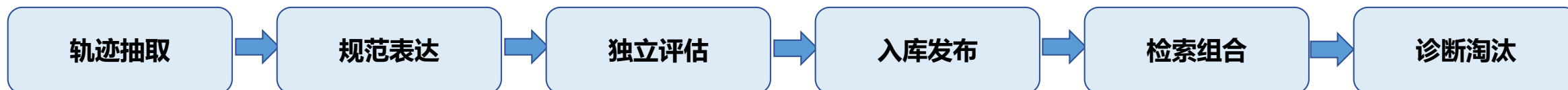
SelfMem: 策略可演化

不预设固定的记忆格式或策略，而是为 Agent 提供记忆工具集和反馈信号，让它自主探索、评估并迭代优化自己的**记忆策略**



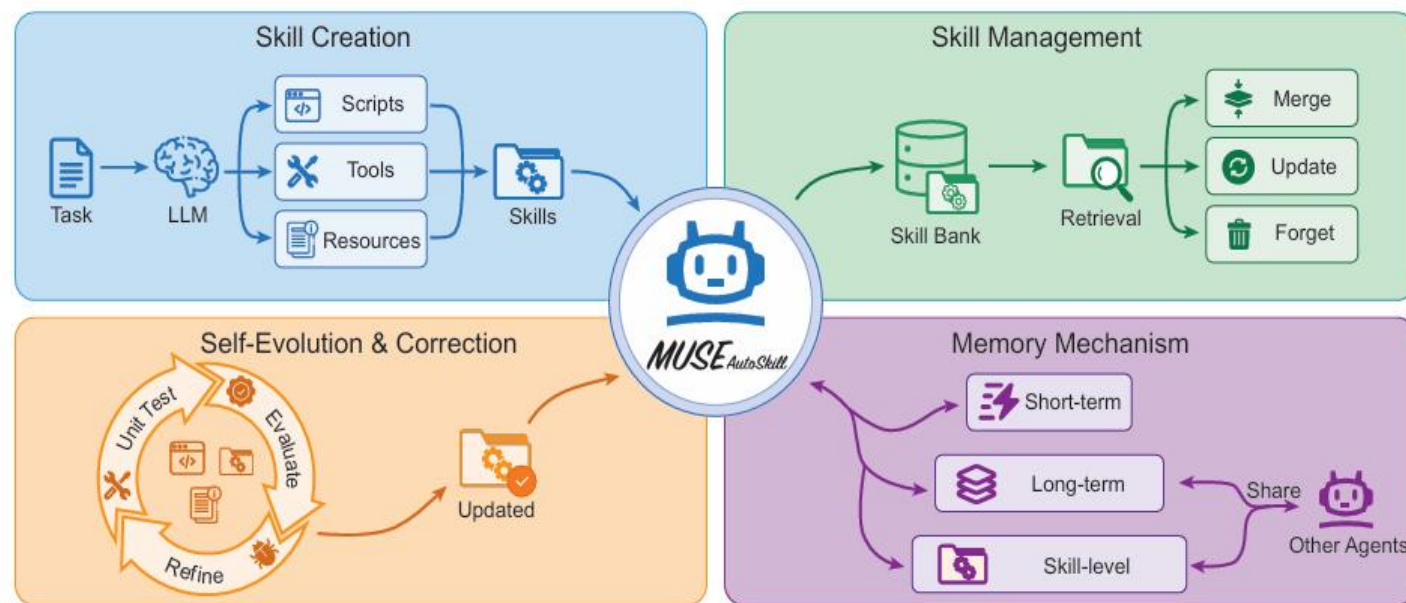
**记忆系统的"进化"不等于"记更多东西",
而是让记忆系统的每一层——操作、架构、策略——都成为可被反馈驱动更新的对象**

Skill: 生命周期



MUSE-Autoskill: Lifecycle

creation —> memory —> management —> evaluation —> refinement 构成统一生命周期：按需创建、跨任务存储、目录检索，并积累 per-skill experience。



MUSE将技能组织为创建、记忆、管理、评估和优化的统一生命周期，使智能体能够随着时间推移，凭借积累的经验生成、优化和复用技能。

Skill 自进化涉及完整生命周期：

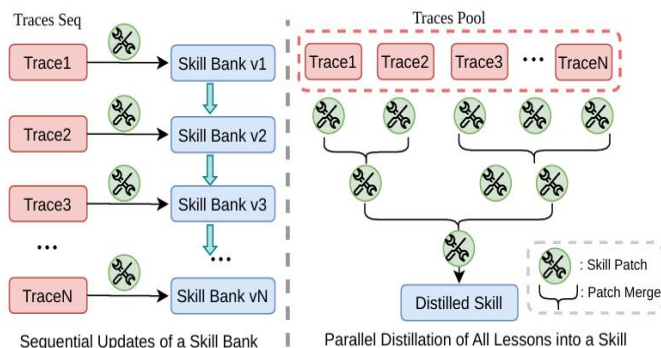
Skill 必须从一次性产物，变成可创建、可使用、可评估、可修订的长期能力资产。

Skill: 进化来源

- Skill 的候选能力可以来自Agent 的执行轨迹，以及外部世界中可验证知识。

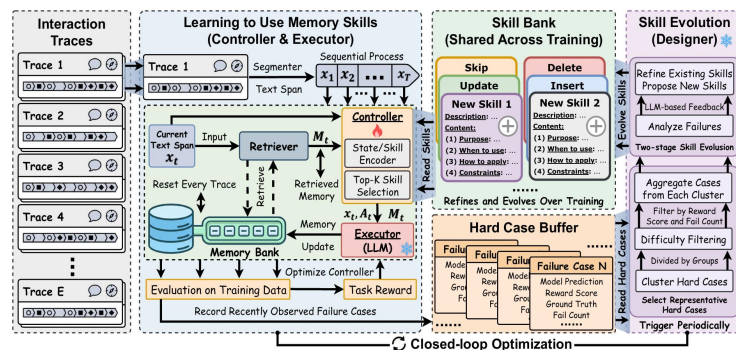
Trace2Skill

从多条**执行轨迹**并行蒸馏 trajectory-local lessons, 形成可迁移的 skill directory.



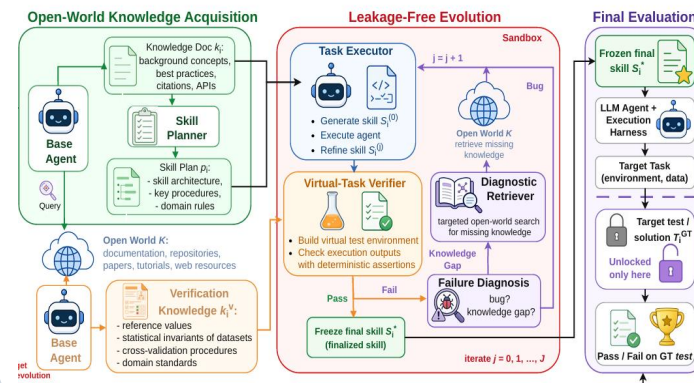
MemSkill

将**交互经验**持续抽象为可复用技能，并通过长期记忆实现技能的积累、检索与演化。



OpenSkill

从**文档**、**代码库**和 **Web** 获得 grounded knowledge 与 verification anchors, 再自建任务改进 skill



Skill: 进化机制

- Skill 进化需要围绕**质量控制、诊断优化、协同进化和元机制**形成持续更新。

SkillAxe:

自诊断与定向修订

质量诊断 → 改进 brief → Skill 重写

将 skill 质量拆为四个可解释维度，先定位问题，再生成定向修订。

SkilIOS:

延迟反馈下的库治理

前序轨迹
验证

Curator → SkillRepo → 后续任务

以连续任务的延迟收益训练 curator，决定经验是否留存于技能库。

SkillRL:

协同进化

经验蒸馏 → 层级 SkillBank ↔ Agent policy

在强化学习中让 SkillBank 与 Agent policy 共同迭代，而非只更新单条 skill。

MetaSkill-Evolve:

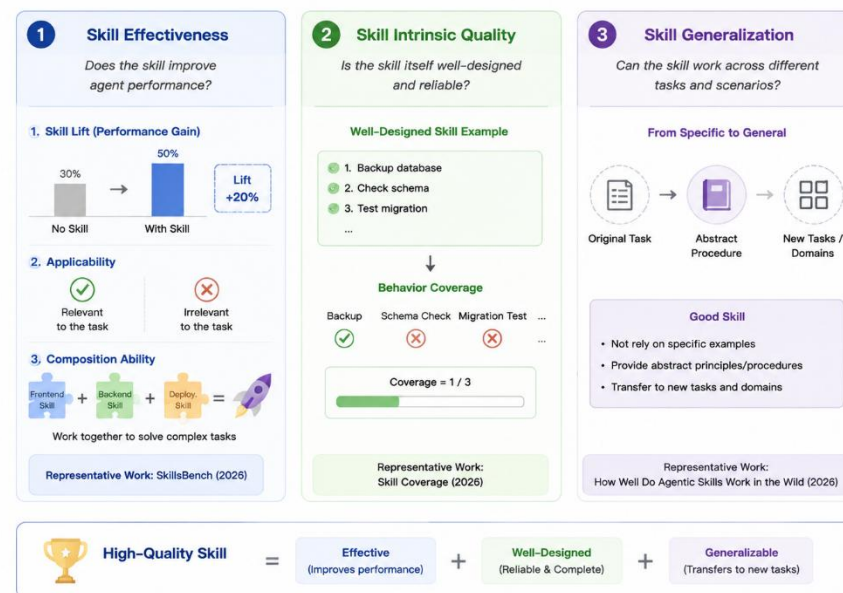
递归元演化

Task skill + Analyze / Retrieve / Propose / Evolve

除 task skill 外，再演化用于分析、检索、分配、提案和更新的元技能。

What Makes a High-Quality Skill?

A high-quality skill should be effective, well-designed, and generalize reliably.



高质量的 Skill 应具备三个核心特征：能够有效提升 Agent 的任务表现；具备内在良好的设计结构与可靠的执行流程；并能够在不同任务和场景中实现有效泛化。

Skill: 优化方法

- 现有SKILL表达问题：多为SOP形式，步骤表达存在冗余、特化、泛化等现象；造成**效果差、效率低**

Type	Example	Effect
Redundant	“Read input.xlsx with pandas and openpyxl ”	Wastes tokens; adds noise
Specialized	“Conditionally clear cells using adjacent/deleted-row data”	Helps one task; may mislead others
Generalizable	“Check answer files with both data_only=False (formulas) and data_only=True (cached values)”	Useful to most tasks

冗余
浪费 token

特化
只适用于某个任务，可能误导其他任务

泛化
有价值，但被其他内容淹没

AtomTip 核心思想：把技能拆成独立的“原子提示（atomic tips）”，每个提示是一句可单独评估有用性并检索的句子

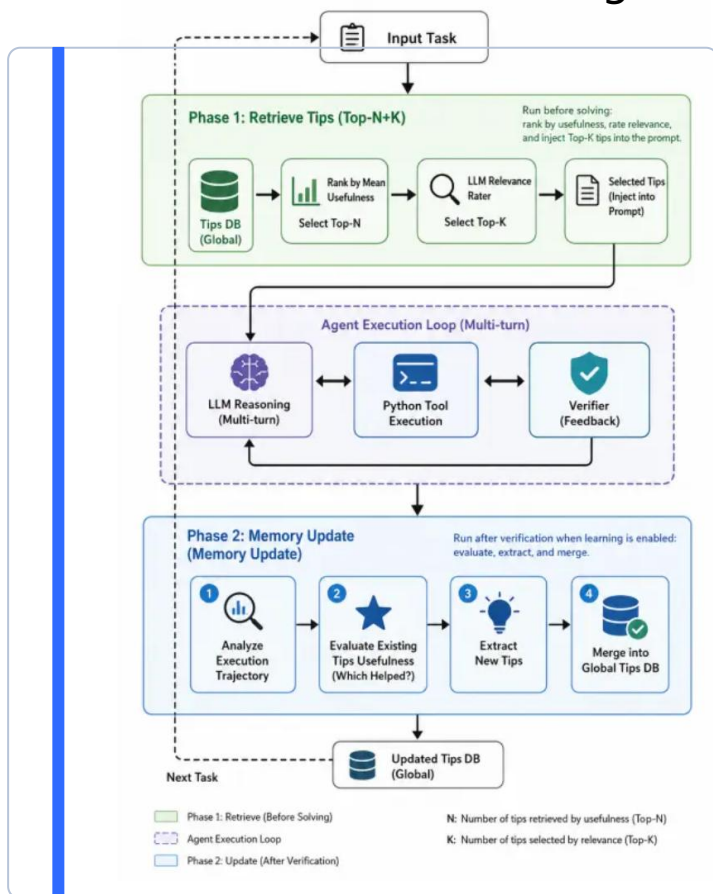
有用性过滤（Usefulness）：对应泛化性，按历史平均分保留 Top-N 候选，排除长期无效的提示

相关性排序（Relevance）：对应特化性，用 LLM 给每个候选打分，取 Top-K 加入当前提示词

Skill: 优化方法

自进化流程:

- **召回提示:** 从全局提示库按历史平均有用性选出 Top-N, 再让 LLM 按当前任务相关性打分, 选出Top-K注入提示词
- **Agent 执行循环:** Agent 结合注入的提示进行多轮推理, 调用 Python 工具执行, 最后通过 Verifier 获取反馈
- **记忆更新:** 任务结束后, agent分析执行轨迹, 评估已用提示的有用性, 提出新提示, 并合并回全局提示库



实验设置	正确率	提升
无skill	48.75%	-
完整SOP	49.25%	+0.5%
原子化提示+语义召回	78.00%	+29.25%
原子化提示+按评分召回	81.75%	+33.0%

实验结论:

- 效果提升主要源于**原子化提示**
- 按评分进行**两阶段筛选**, 相比语义召回可进一步提升效果

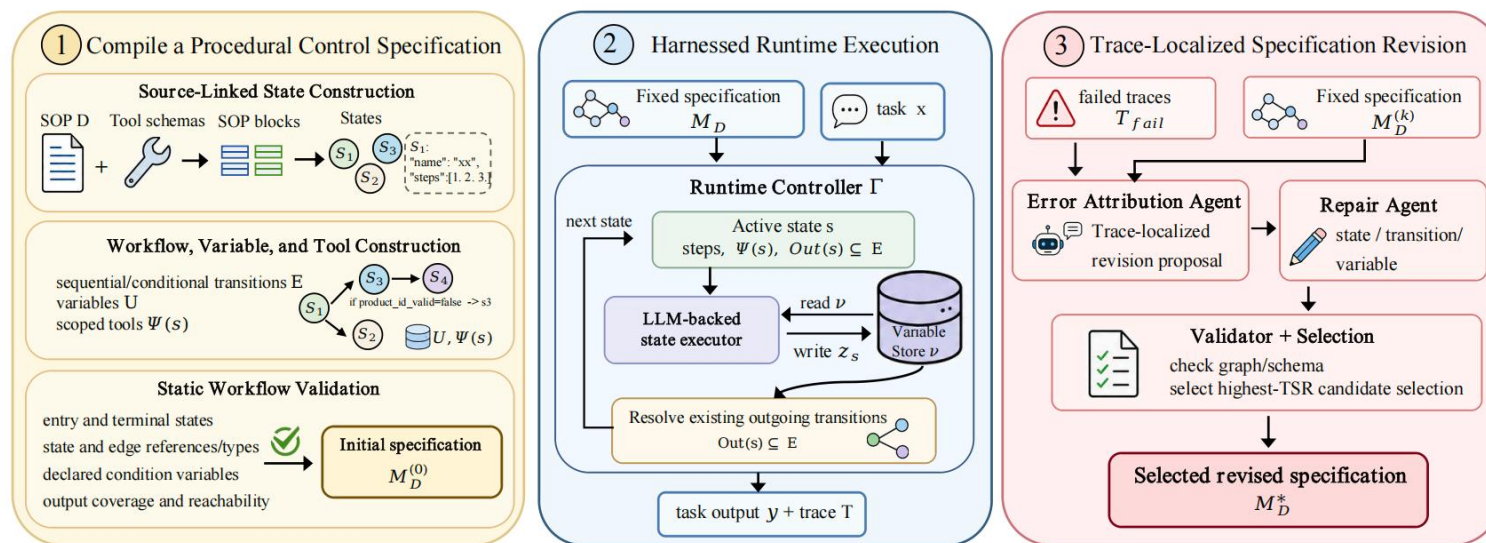
Skill: 优化方法 (harness)

- 重复 SOP 任务中, Agent 不应每次**重新读流程**, 而应把 SOP 编译成**可复用、可执行、可修订**的控制层。

流程控制无法稳定复用

失败经验没有明确写回

重复任务成本高、不稳定



SOPFlow 的方法是把自然语言 SOP 和工具 schema 编译成一个可执行的流程控制规格, 再由共享 runtime controller 按状态执行, 并用失败轨迹反向修订流程。

SOPFlow 把流程知识从“上下文提示”提升为“可复用控制层”。

Skill: 优化方法

- 复用收益：一次构建和修订，可以被**多次执行摊销**

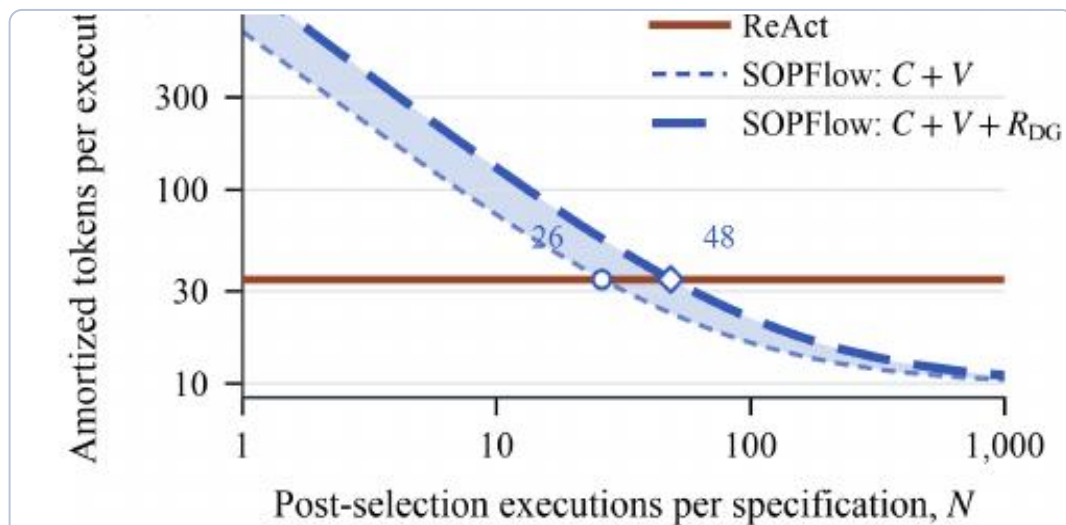


Figure 4: Average amortized tokens per execution over

平均摊销 token 曲线

break-even

只计 construction + validation 时，约 26 次执行后低于 ReAct。

含修订成本

加入 DG 校准修订成本后，约 48 次执行后达到摊销优势。

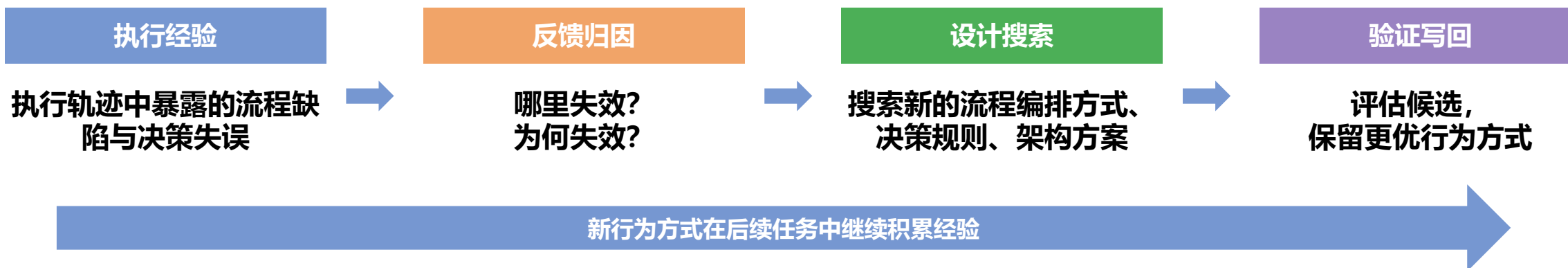
长期下限

复用次数增加后，平均成本逼近 SOPFlow 的 9.8K 在线 token floor。

当同一 SOP 反复执行时，固定规格逐渐转化为成本优势

Workflow / Policy Evolution

- 核心问题: **Agent 如何改变自己的行为方式?**
- Agent不只是学会新Skill, 还会主动优化自己 **“做事的流程和决策逻辑”**



AgentEvolver

Experience-driven Policy Evolution

从任务轨迹中发现不足; 以自提问、自导航、自归因, 把经验转成下一轮探索与学习策略。

AFlow

Workflow-level Evolution: 代码化 workflow 的 MCTS 搜索

将 workflow 视为搜索空间, 借助代码修改、树状经验与执行反馈自动生成、评估和优化 pipeline。

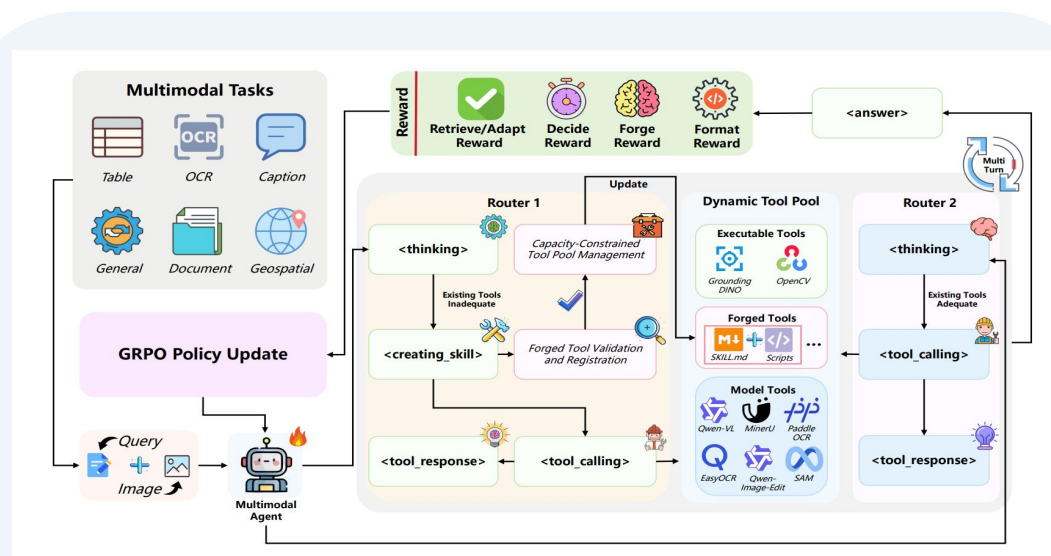
ADAS

Agent-level Evolution: Meta Agent Search 自动设计系统结构

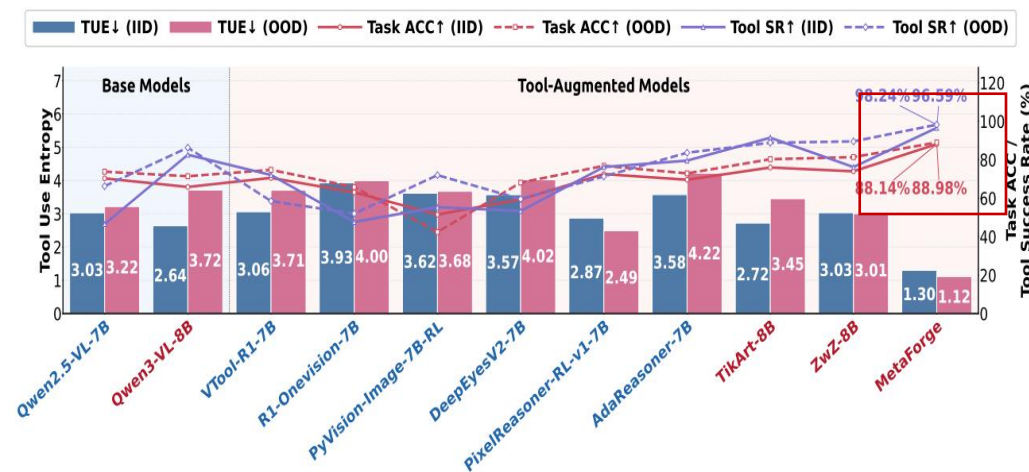
在代码空间中发明或组合新的 agent component、prompt、tool use 与 control flow, 优化整体性能。

工具

- 静态工具库难覆盖新 GUI、新文档和跨模态任务，Agent 难以自行发现能力缺口。
- 以任务轨迹触发“需求识别—工具检索—适配/生成—执行验证—能力写回”闭环。
- 工具自进化不仅是多装工具，工具的有效组合使用方式。



- 工具“检索—适配—生成—回收”闭环，将一次性工具调用扩展为长期能力写回机制。



- 工具自进化的核心收益，不是单次调用成功率提升，而是系统执行能力边界的持续扩展

User Model: 画像与认知状态驱动 Agent 自进化

- 用户画像不仅用于存储用户信息，更应持续驱动Agent决策，并依据交互反馈不断演化。

01 O-Mem: 主动更新用户画像

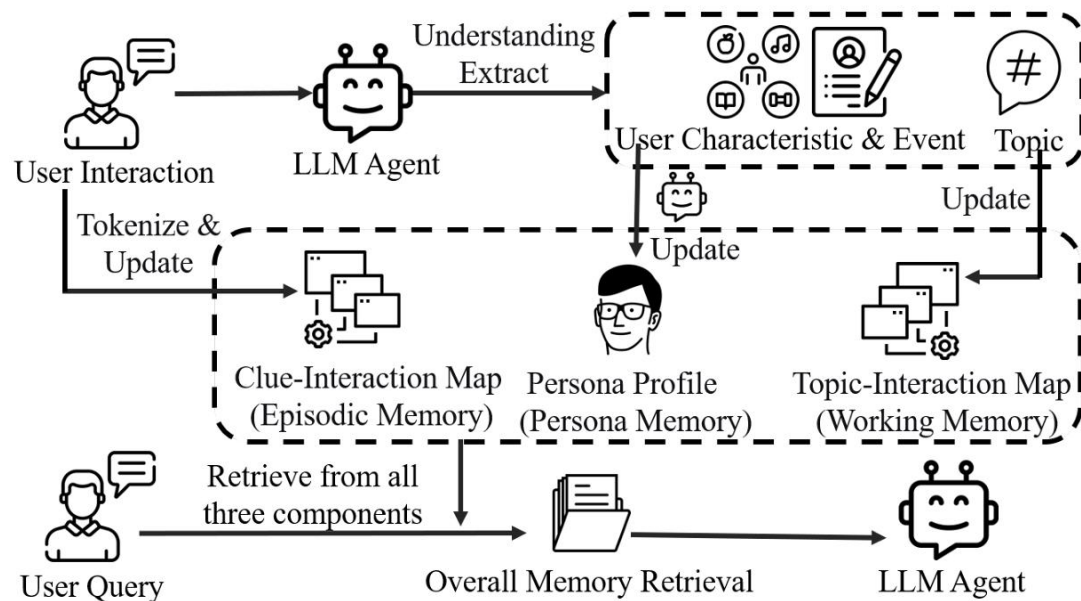
- 从长期交互中动态提取用户特征和事件记录，并分层检索 persona 与主题上下文；画像成为长期交互资产。

02 RecNet: 偏好更新也进入策略自进化

- router agent 整合并传播实时偏好；基于反馈优化偏好传播策略，filter memory 控制选择性吸收。

03 GazeMind: 认知负荷驱动交互适配

- 将眼动转成结构化表示以推断认知负荷，并结合用户特征与历史参考实现个性化适配



将用户交互编码至记忆时，提取相关用户属性与事件数据，分别存入人格记忆、情节记忆和工作记忆（不同颜色代表不同记忆组件）。面对新用户查询时，O-Mem从上述三个记忆组件中并行检索相关信息。

画像不只记录**偏好**，也可记录**目标**、**掌握度**与**认知负荷**；这些不确定信号应被转成可验证、可撤销的策略更新。

Training : 经验持久化为参数更新

- 自进化的参数更新, 关键是 agent 能否以自身经历产生训练数据、监督或任务。

内化参数能让智能体在连续交互中不断提炼共性规律, 形成稳定的决策偏好, 无需每次外部临时检索

RQ1: 如何根据失败反馈生成训练数据?

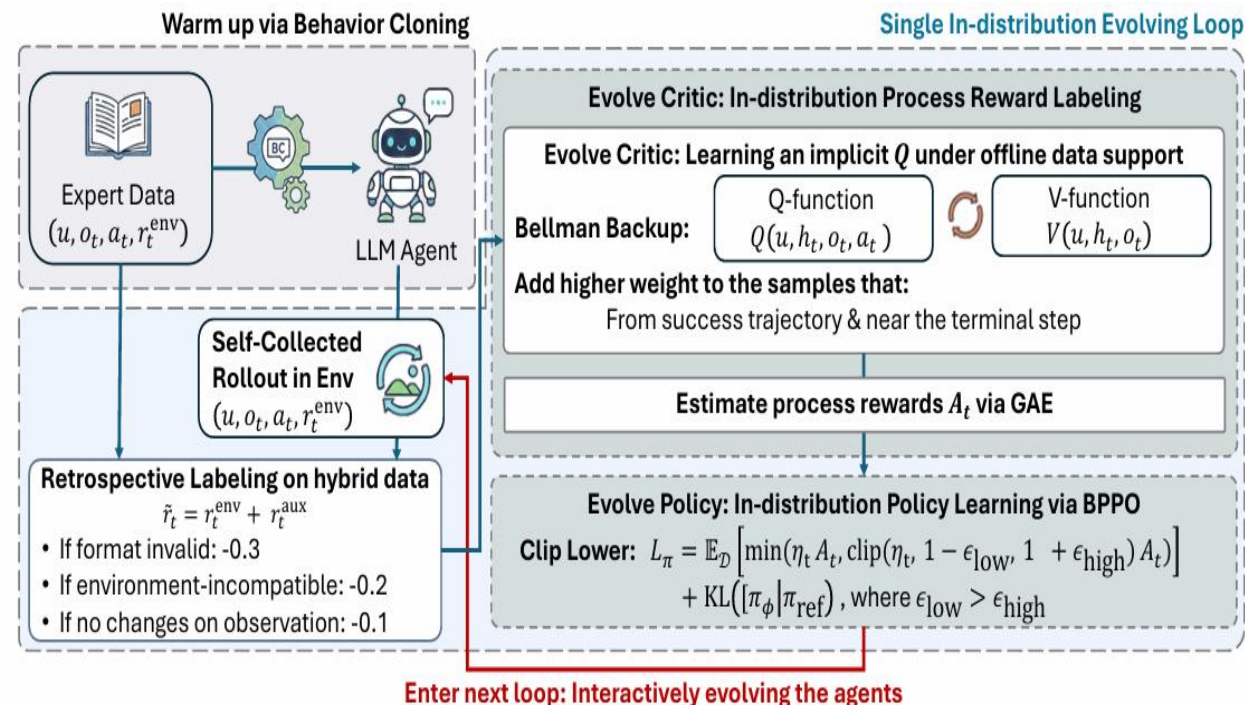
- TT-SI: 识别不确定样本 -> 生成相似样本
-> test-time fine-tuning

RQ2: 如何将长程轨迹变成过程监督?

- Q-Evolve: expert + agent trajectories -> in-distribution critic -> process rewards + policy update

RQ3: 如何根据结果筛选价值训练数据?

- RePro: 任务完成后评估并筛选高价值步骤, 据此使用 RePro-PO 训练 agent。



Q-Evolve先行为克隆预热策略, 再经多轮分布内循环迭代。每轮混合专家与自采数据构建缓冲池, 回溯标注奖励, 经贝尔曼传播近似max-Q, 导出步级优势并下放至词元级实现优化。策略、评论家与数据集闭环协同演化, 更新均约束于当前轮次分布内。

任务、课程与环境，形成持续学习闭环

- Agent 自造可验证任务、课程或环境，打造持续训练闭环，驱动持续学习。

01

Self-Play SWE-RL

自造可验证任务

- 真实代码库
- 注入 bug <-> 修复 bug
- test patch 驱动 RL 更新

02

SAGE

自造课程与训练信号

- Challenger 生成任务
- Planner / Solver / Verifier
- Critic 筛选轨迹 -> 更新共享模型

03

Agent-World

自造环境与目标任务

- 探索数据库 / 工具生态
- 合成可验证、可控难度任务
- 按能力缺口驱动持续训练

Agent 持续构建可验证的学习环境（任务、课程或开放环境），驱动自身进入下一轮能力演化

Agent与环境共进化

- 自进化要求 Agent 与环境互为进化压力：对世界的理解要进化，评估标尺也要跟着进化。

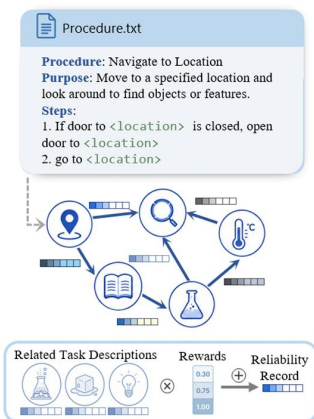
向内：Agent 对环境认知的进化

ProPlay: Procedural World Models

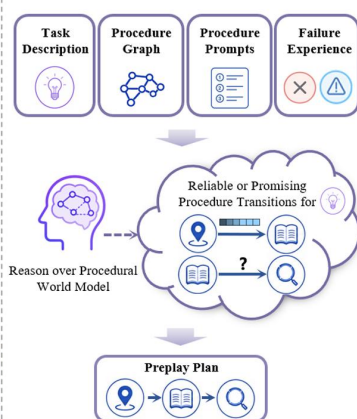
执行轨迹 → Procedure Graph → Preplay 预演 → 反馈精炼

将成功轨迹抽象为过程图，执行前模拟未来程序路径，执行后按环境反馈更新图结构与可靠性记录。

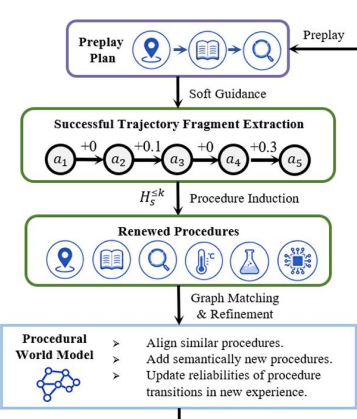
a Procedural World Model



b Procedure-Level Preplay



c Episodic Refinement

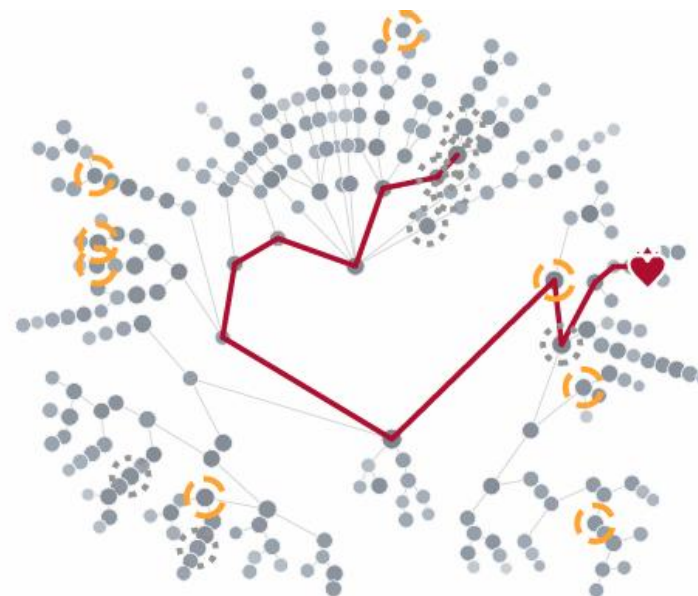


向外：进化环境对 Agent 的要求

Red Queen Gödel Machine

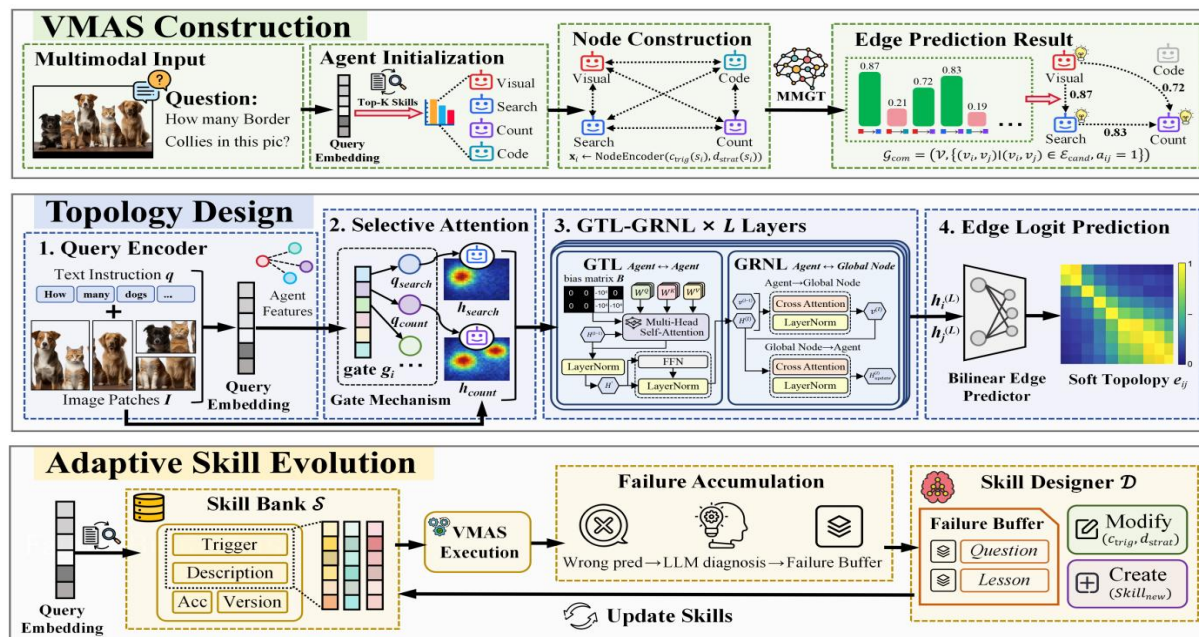
Agent 变强 → 标尺更难 → 新能力受检验 → 持续进化

将评估器纳入循环：epoch 内保持标准固定，在边界处以 ground-truth anchor 推动效用与评估器升级。



协作结构自进化

- 固定的 planner-executor-reviewer 拓扑 **不适合复杂多模态任务**
- 不同图像、不同任务需要不同的**专家组合**和**通信路径**。
- 结构自进化意味着：系统不仅学习“谁会什么”，还学习“**谁应该和谁交流**”。
- 这是从节点能力进化走向**图结构**进化。



- 通过节点构建、边预测与拓扑重组，将多智能体系统从固定分工结构升级为可按任务动态组织的协作图。

Model	Method	MMBench	MathVista	RealWorldQA	InfoVQA	Average
Qwen3-VL-8B-Instruct	Direct Answer	84.2	77.3	71.4	83.2	79.0
	Linear	83.7	79.7	73.4	82.7	79.9
	+ SkillGraph	85.8 $\uparrow 2.1$	80.8 $\uparrow 1.1$	74.4 $\uparrow 1.0$	84.2 $\uparrow 1.5$	81.3 $\uparrow 1.4$
	Layered	84.6	80.2	73.9	82.9	80.4
	+ SkillGraph	86.3 $\uparrow 1.7$	81.5 $\uparrow 1.3$	75.4 $\uparrow 1.5$	84.5 $\uparrow 1.6$	81.9 $\uparrow 1.5$
	Centralized	84.3	80.5	73.2	82.8	80.2
	+ SkillGraph	86.1 $\uparrow 1.8$	82.2 $\uparrow 1.7$	74.9 $\uparrow 1.7$	84.1 $\uparrow 1.3$	81.8 $\uparrow 1.6$
	Random	85.2	81.6	75.3	84.9	81.8
	+ SkillGraph	86.6 $\uparrow 1.4$	82.8 $\uparrow 1.2$	76.1 $\uparrow 0.8$	85.4 $\uparrow 0.5$	82.7 $\uparrow 1.0$
	Complete	84.9	81.8	75.2	85.0	81.7
	+ SkillGraph	86.8 $\uparrow 1.9$	83.2 $\uparrow 1.4$	76.4 $\uparrow 1.2$	85.3 $\uparrow 0.3$	82.9 $\uparrow 1.2$

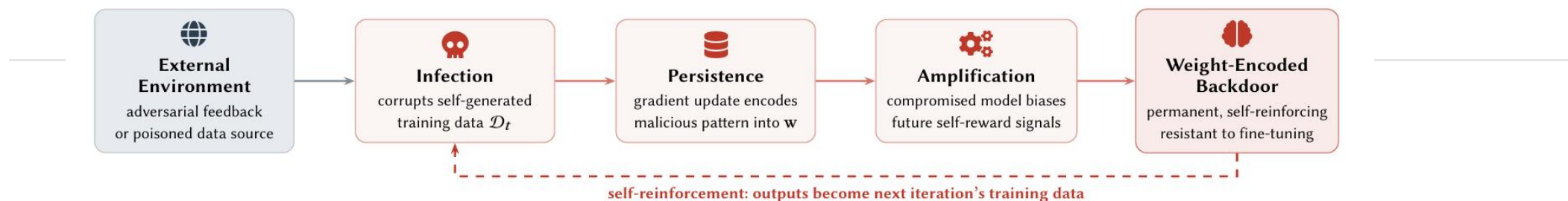
进化的不只是专家节点，还有它们之间的连接方式

Safety: 安全机制

- 当改进会进入 memory / skill / tool / policy 等长期资产时，攻击影响与行为偏置不再止于单次会话。

01 Safety in Self-Evolving: 攻击面拓展至进化全生命周期

- MLAS 将攻击面分为 5 个功能模块 x 5 个生命周期阶段；攻击影响可能被**持久编码**、**跨代放大**，并在人群中传播。



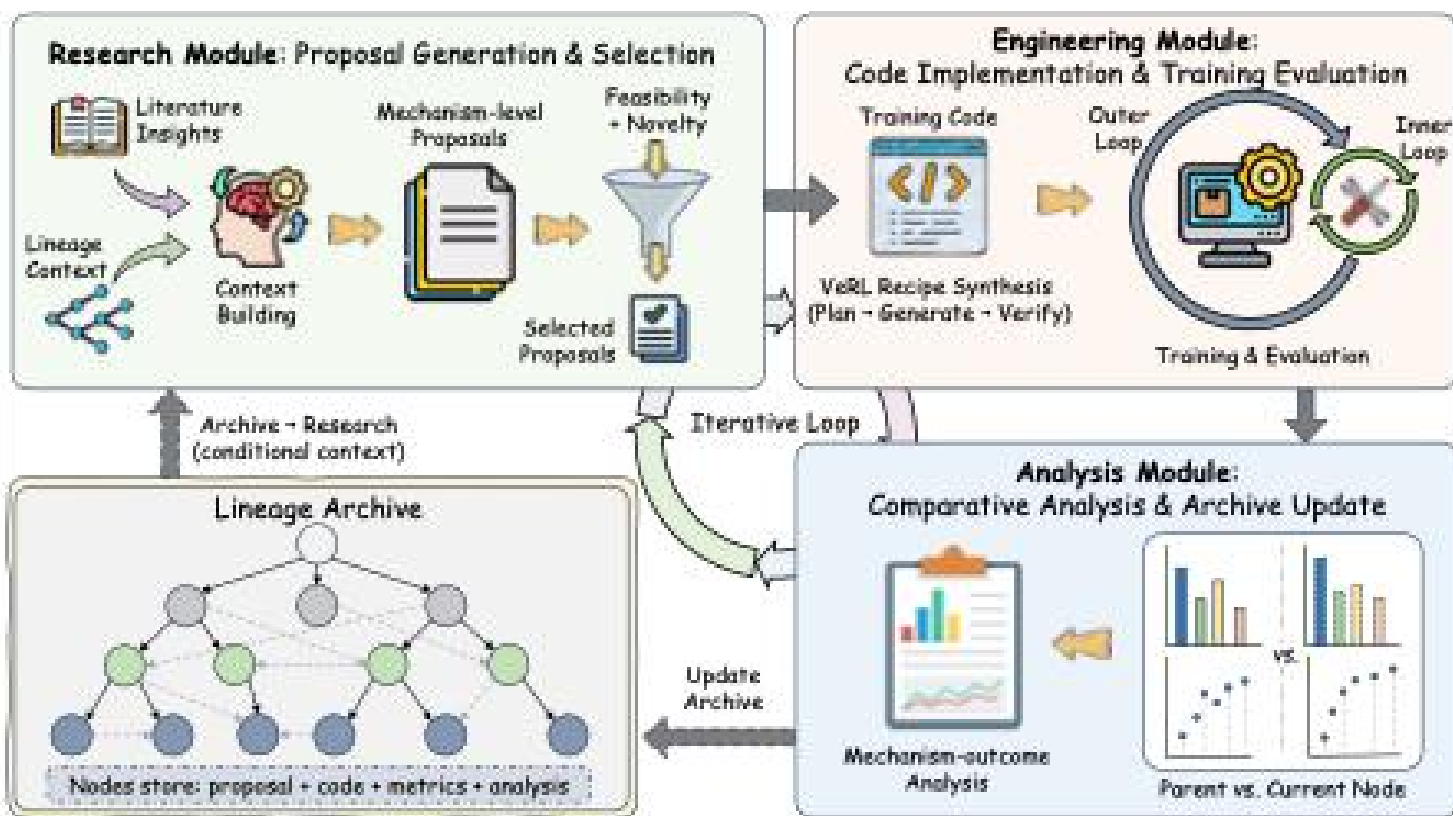
02 On Safety Risks: 良性经验也可能带来安全风险

- 仅从良性任务积累的经验也会带来安全风险（比如，乐于助人的良性经验，更难拒绝危险请求）；补充拒答经验可缓解下降，却可能引发 over-refusal。

安全管控应覆盖自进化智能体整个生命周期，而不是仅仅在训练或部署之后再补一道安全防线。

自主算法创新

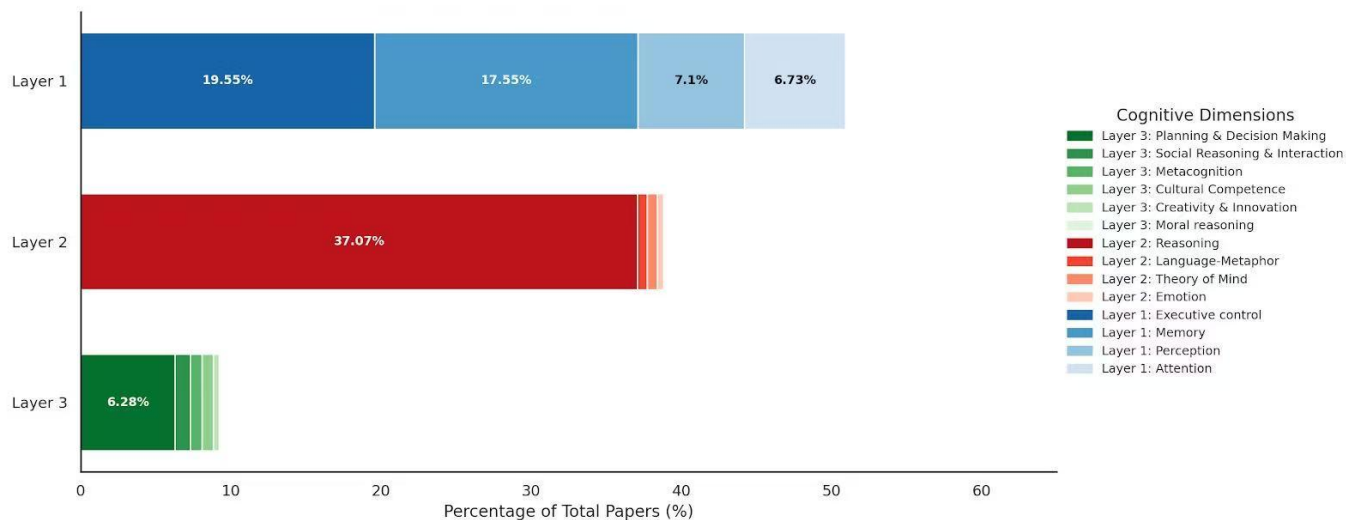
- 自主算法创新过程基本可行
 - 以LLM训练强化学习算法的自主创新为例
 - Phase 1: Idea Proposal
 - Phase 2: Implementation, verification, and evaluation
 - Phase 3: Reflective analysis with archive update



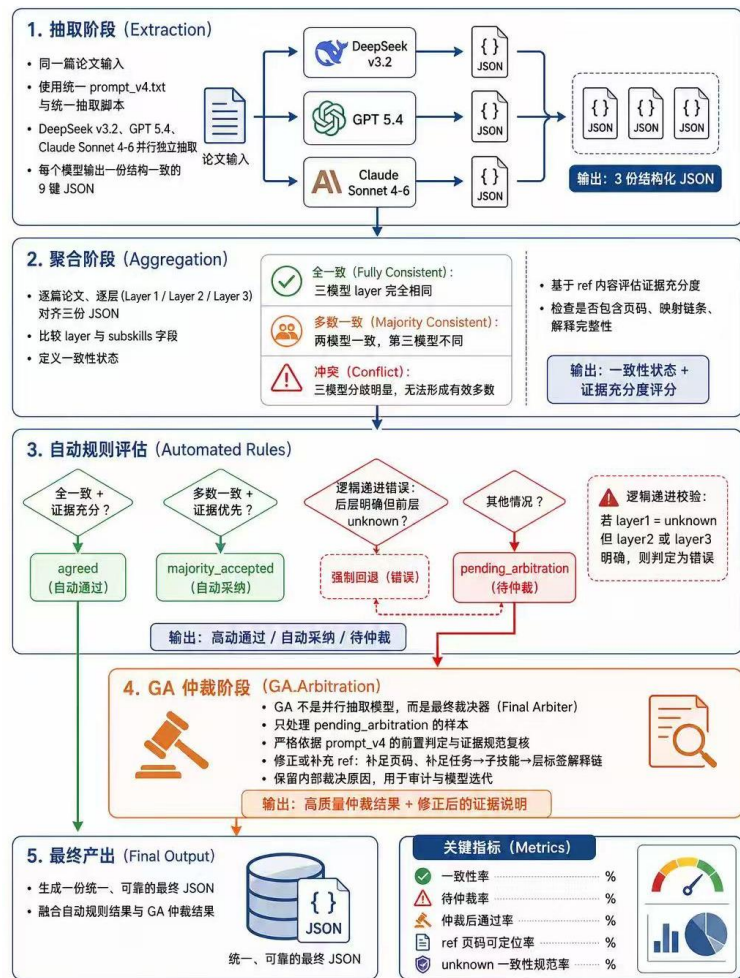
自主文献调研

- 针对4000篇人类认知评测文献的自主调研、自主分析，为大模型认知评测提供指导，开展对比分析，为后续大模型自动化 Benchmarking提供依据

Figure 1: Global Proportion of Cognitive Dimensions (Total %)



✓ 分析结果

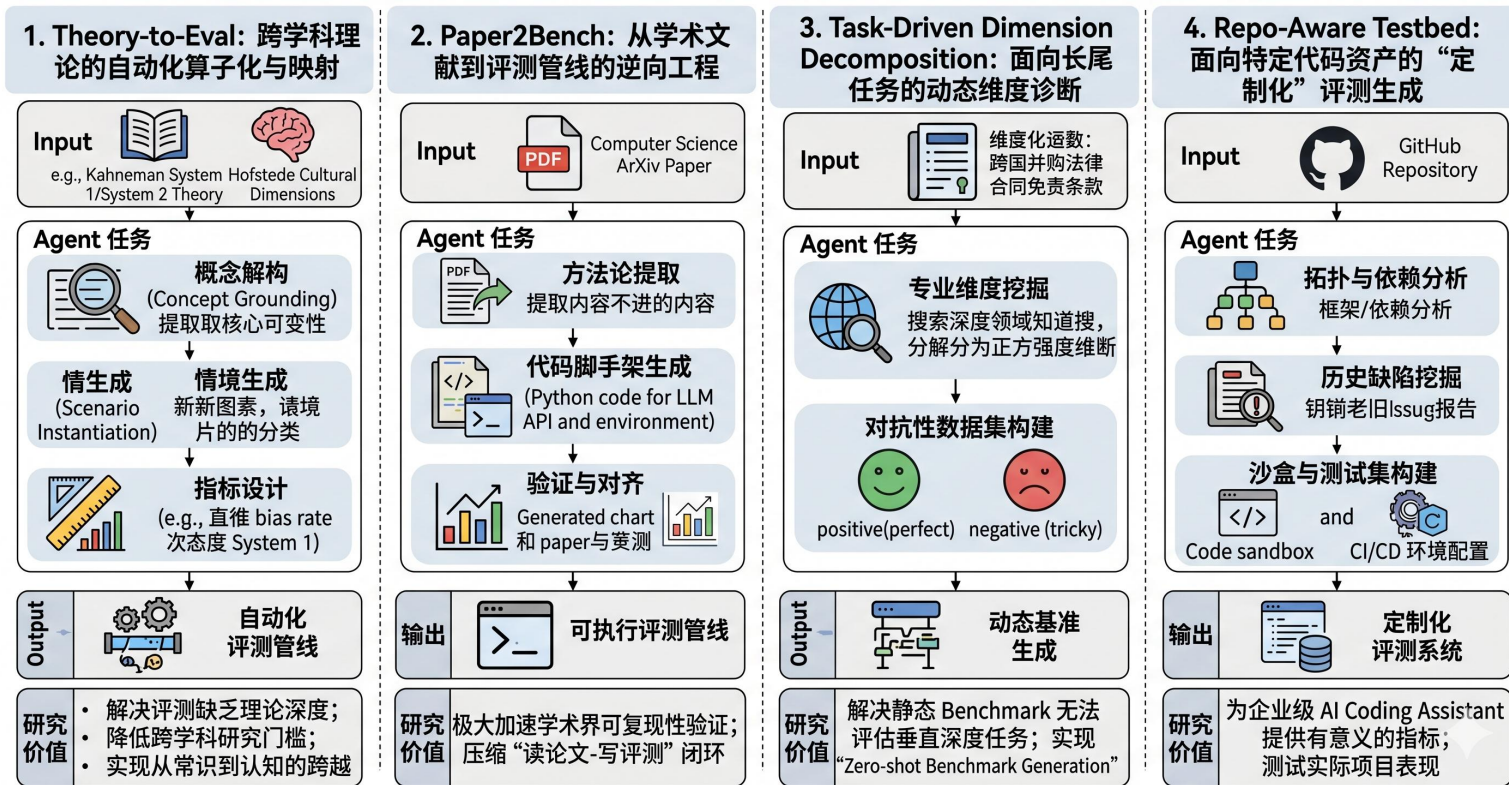


✓ 文献处理流程

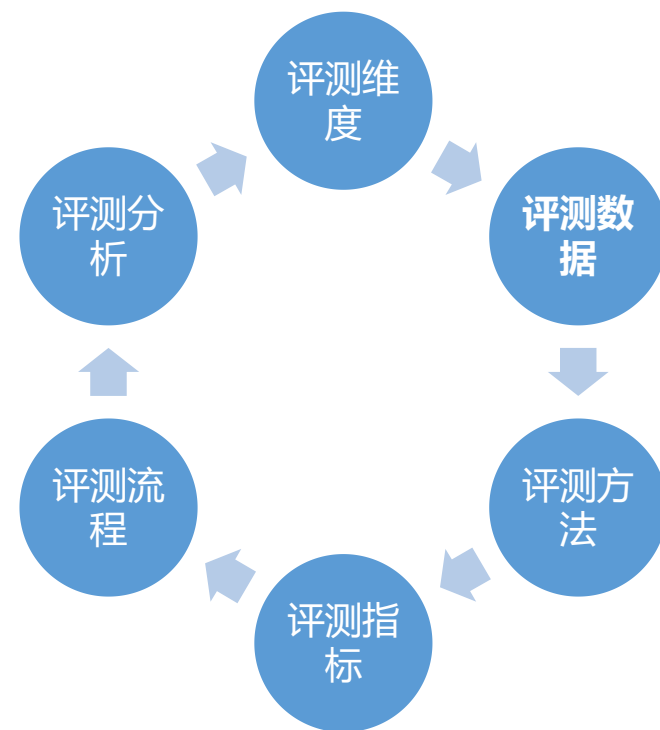
Benchmarking自动化

- 根据**人文学科理论、学术文献、任务描述、代码仓库**，自动生成评测数据、指标、方案

EXAMPLE INSTANCES OF THE AUTOMATED EVALUATION FRAMEWORK



Benchmarking Automation的基本流程



Benchmarking的核心要素

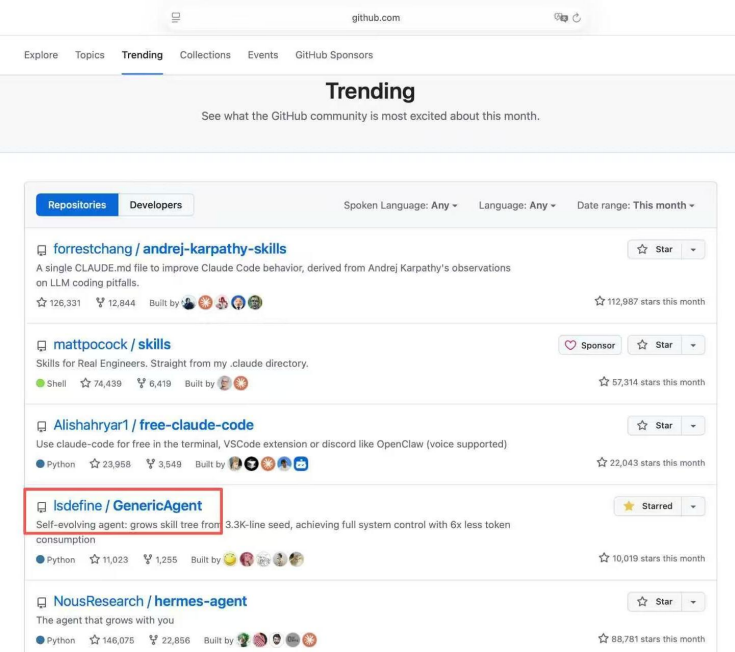
自进化智能体-GA



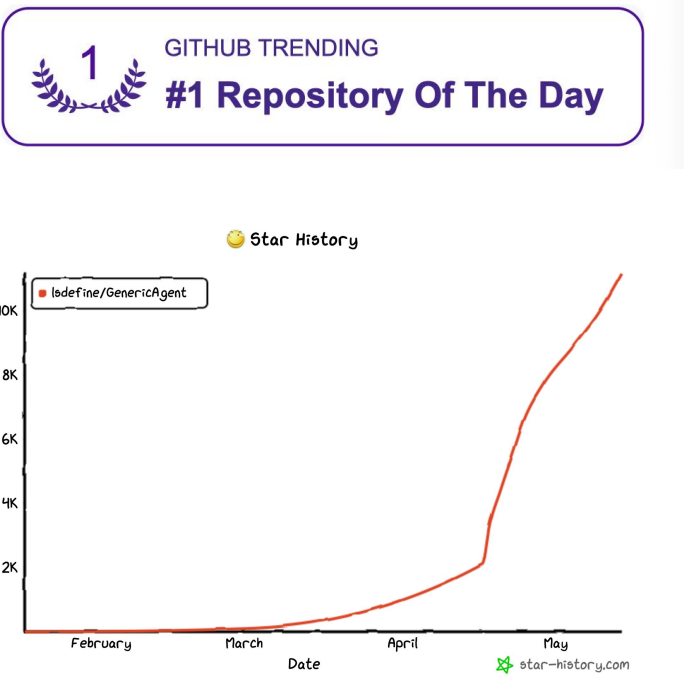
龙虾之躯：自进化智能体

● Generic Agent vs OpenClaw

Comparison	Generic Agent	OpenClaw
Code Scale	Core ~3,000 lines, fully auditable end-to-end	~400,000-500,000 lines (excluding plugins), high audit cost
Learning Capability	PERSISTENT MEMORY AUTONOMOUS LEARNING, GETS BETTER WITH USE	BUILT IN MEMORY MODULE, RELIES ON PLUGINS FOR EXTENSIBILITY
Tool System	9 ATOMIC TOOLS FREELY COMPOSABLE, ZERO PLUGIN DEPENDENCY	DEPENDS ON PLUGIN MARKETPLACE, PLUGIN QUALITY VARIES
China Scenario	DEEPLY ADAPTED TO WECHAT / ALIPAY / GOVERNMENT OA, ETC.	PRIMARILY FOCUSED ON OVERSEAS SCENARIOS
Security	MINIMAL ARCHITECTURE, FULL OPERATION TRACEABILITY	OPEN PLUGIN ECOSYSTEM, UNIFIED SECURITY BOUNDARIES ARE HARD TO ENFORCE
Deployment	ONE STEP DEPLOYMENT, NO PROFESSIONAL TECHNICAL TEAM NEEDED	REQUIRES TECHNICAL TEAM FOR CONFIGURATION AND MAINTENANCE



2026 年 4 月github趋势榜全球第四，力压hermes agent，仅次于claude code



Reached #1 on GitHub Global Trending, with 13.2K stars.

极简架构

- 代码越多 ≠ 功能越强；极简才能高效进化、才能灵活适应、才能智能涌现

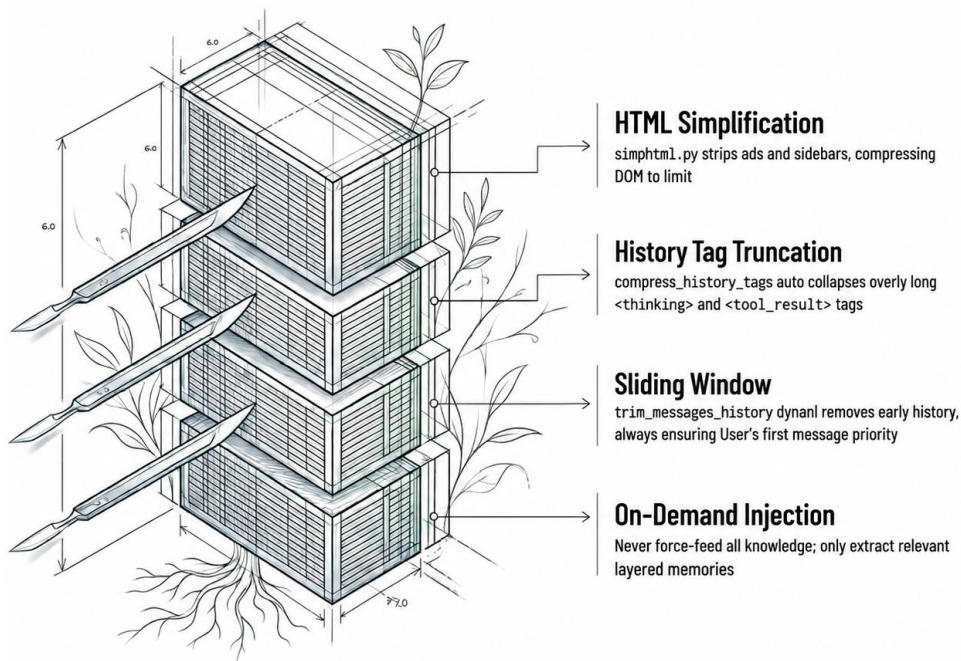


“如无必要，勿增实体”，即“简单有效原理”
——Occam's Razor（奥卡姆剃刀定律）

- 只有当架构小到足够透明，智能体才可能从“会用系统”走向“会改系统”，最终实现自主进化

极致压缩

- 保持一个轻量、敏捷、高信息密度的决策大脑，不要让智能体被自己的历史包袱与无效信息拖垮



通过逐层的过滤与压缩，使用 <30K 的上下文长度，达到传统框架200K-1M的效果

The screenshot shows a web page with the title "AI Agent 场景下，万级 JSON 字段的性能挑战与" (Performance Challenges and Solutions for Tens of Thousands of JSON Fields in AI Agent Scenarios). The page includes a sidebar with a user profile for "SelectDB技术团队" (SelectDB Technical Team) and a main content area with a detailed article. A dark overlay box on the right side of the page provides a comparison of token usage:

- 压缩可量化 - 显性对比版
- 彩色框 = 对LLM低价值、会被压缩/降权的区域
- 蓝侧栏/目录/热门 橙广告 紫相关推荐
- 这页真实压缩主要来自: script/style/长属性清理 + 侧栏/推荐/广告等低价值DOM + Python二次token优化
- optHTML Saved=80.8%, 极限版Saved=91.2%
- 当前高亮 24 个区域

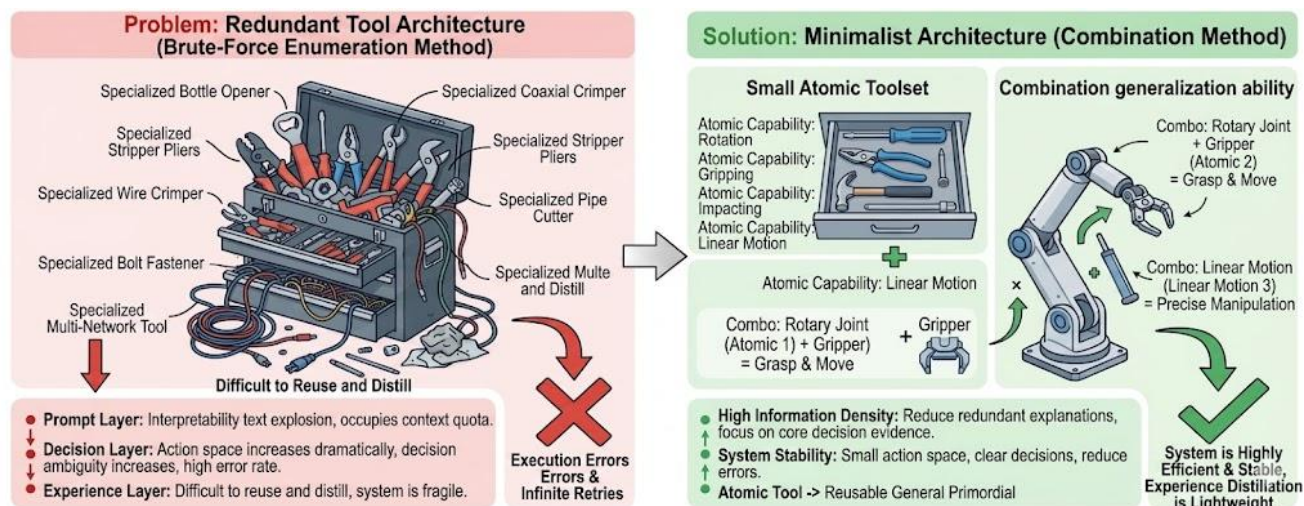
The article text discusses the performance challenges of handling large JSON fields in AI Agent scenarios, mentioning the use of Doc Mode and Segment V3 for optimization. It also includes a list of typical application scenarios and a comparison of token usage between different methods.

网页包含大量低价值区域：侧栏、广告、推荐、评论、登录组件等。通过无效信息压缩，节省约 90%的Token

最少工具

● 工具冗余是智能体的“隐性杀手”

- Prompt Overload: 每一个新增工具的说明文件都会挤占宝贵的上下文额度
- Policy Collapse: 工具越多，模型的动作空间就越大，决策难度越大



Model	Method	I1-Inst.		I1-Tool		I1-Cat.		I2-Inst.		I2-Cat.		I3-Inst.		Average	
		Pass	Win	Pass	Win	Pass	Win	Pass	Win	Pass	Win	Pass	Win	Pass	Win
ChatGPT	ReACT	41.5	-	44.0	-	44.5	-	42.5	-	46.5	-	22.0	-	40.2	-
	DFSDT	54.5	60.5	65.0	62.0	60.5	57.3	75.0	72.0	71.5	64.8	62.0	69.0	64.8	64.3
Claude-2	ReACT	5.5	31.0	3.5	27.8	5.5	33.8	6.0	35.0	6.0	31.5	14.0	47.5	6.8	34.4
	DFSDT	20.5	38.0	31.0	44.3	18.5	43.3	17.0	36.8	20.5	33.5	28.0	65.0	22.6	43.5
Text-Davinci-003	ReACT	12.0	28.5	20.0	35.3	20.0	31.0	8.5	29.8	14.5	29.8	24.0	45.0	16.5	33.2
	DFSDT	43.5	40.3	44.0	43.8	46.0	46.8	37.0	40.5	42.0	43.3	46.0	63.0	43.1	46.3
GPT4	ReACT	53.5	60.0	50.0	58.8	53.5	63.5	67.0	65.8	72.0	60.3	47.0	78.0	57.2	64.4
	DFSDT	60.0	67.5	71.5	67.8	67.0	66.5	79.5	73.3	77.5	63.3	71.0	84.0	71.1	70.4
Vicuna	ReACT & DFSDT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
	ReACT & DFSDT	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
Alpaca	ReACT	25.0	45.0	29.0	42.0	33.0	47.5	30.5	50.8	31.5	41.8	25.0	55.0	29.0	47.0
	DFSDT	57.0	55.0	61.0	55.3	62.0	54.5	77.0	68.5	77.0	58.0	66.0	69.0	66.7	60.0
ToolLLaMA	DFSDT-Retriever	64.0	62.3	64.0	59.0	60.5	55.0	81.5	68.5	68.5	60.8	65.0	73.0	67.3	63.1

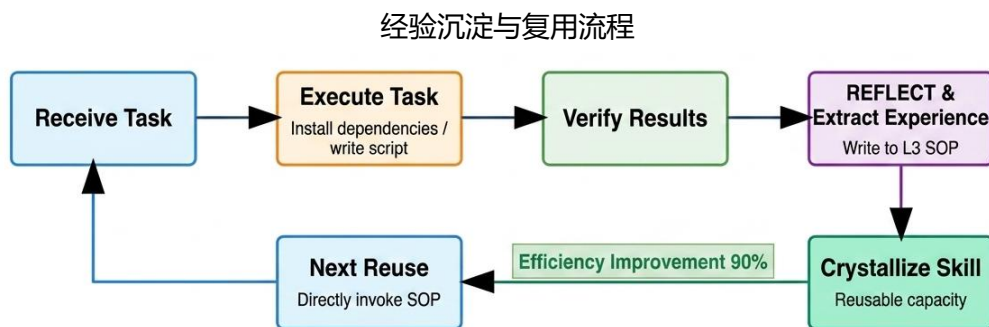
不选择工具，直接暴露16000+工具给模型，模型表现极差[1]

[1] Toolllm: Facilitating large language models to master 16000+ real-world apis." ICLR. 2023.

■ 少即是多：能力源于组合，而非穷举

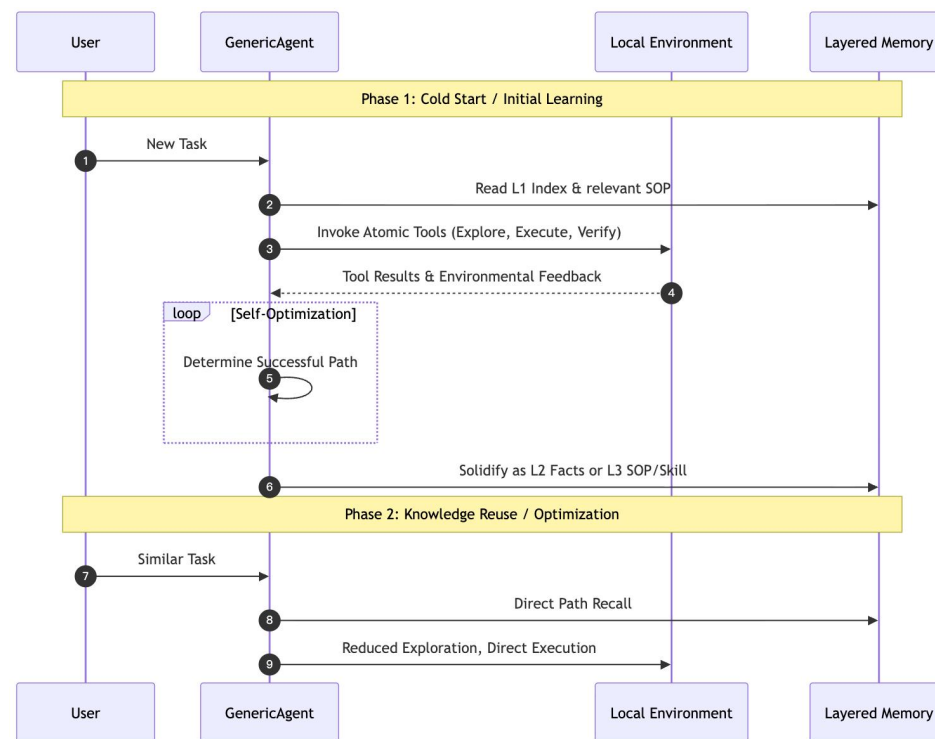
经验进化

- 自我进化能力典型体现：“记住经验、提取经验、应用经验”



进化铁律

1. **无行动，不记忆：** 没验证过的信息不许往里写
2. **神圣不可删改：** 验证过的数据重构时不能丢
3. **禁止存储易变状态：** 时间戳、Session ID 不存
4. **最小充分指针：** 上层只留最短标识，多一个词都冗余



不同层次的记忆在复用流程中的不同角色

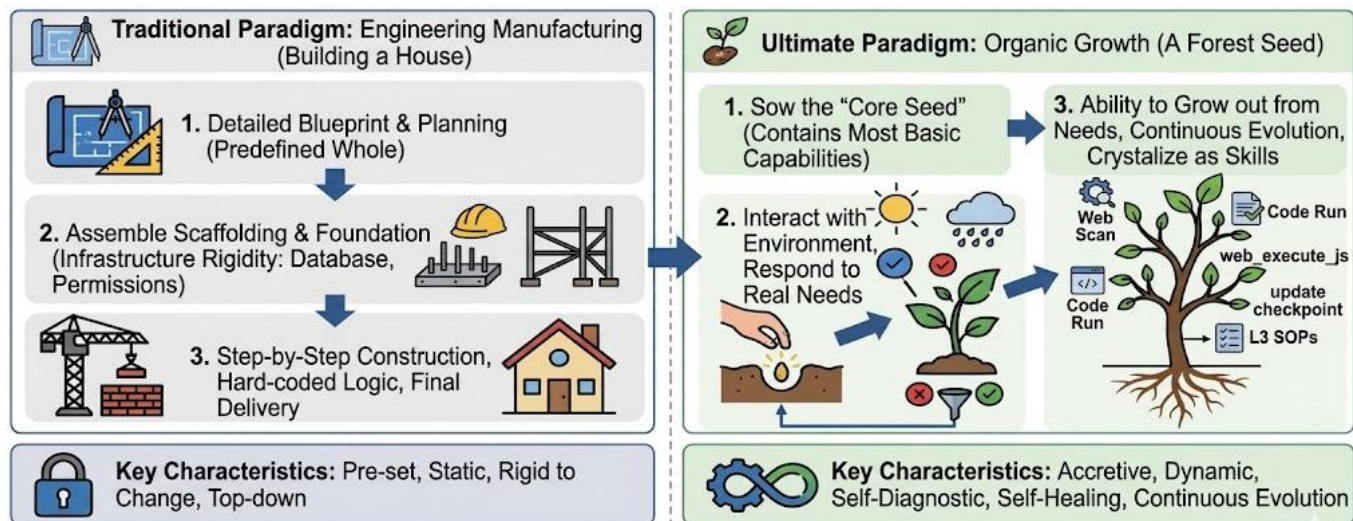
“我们并不是从经验中学习，而是从对经验的**反思**中学习。” ——约翰·杜威 (John Dewey)

能力生长

● 能力不是设计出来的，而是在真实需求中长出来的

Manny Lehman proposed the laws of software evolution (**Lehman's Laws**) :

"As long as a software system is in active use, it must undergo continuous change. Otherwise, it will become increasingly difficult to use and eventually be phased out."



Agent: 这个任务需要查行情数据。我搜索下相关 Python 库。

→ [web_scan]

Agent: 找到 mootdx 这个库。先装一下。

→ [code_run: pip install mootdx]

Agent: 写一段选股逻辑。

→ [file_write: stock_monitor.py]

Agent: 跑起来看看行不行。

→ [code_run: python stock_monitor.py]

Agent: 报错了，接口参数不对，修一下。

→ [file_patch: stock_monitor.py]

Agent: 再跑。OK，正常输出了。

Agent: 配置一个定时任务，每天早上 9:30 跑。

→ [file_write: cron 配置]

Agent: 测试提醒渠道。OK。

Agent: [start_long_term_update] 把这套流程存成一个 Skill。

GenericAgent “长出来的” 监控股票的流程

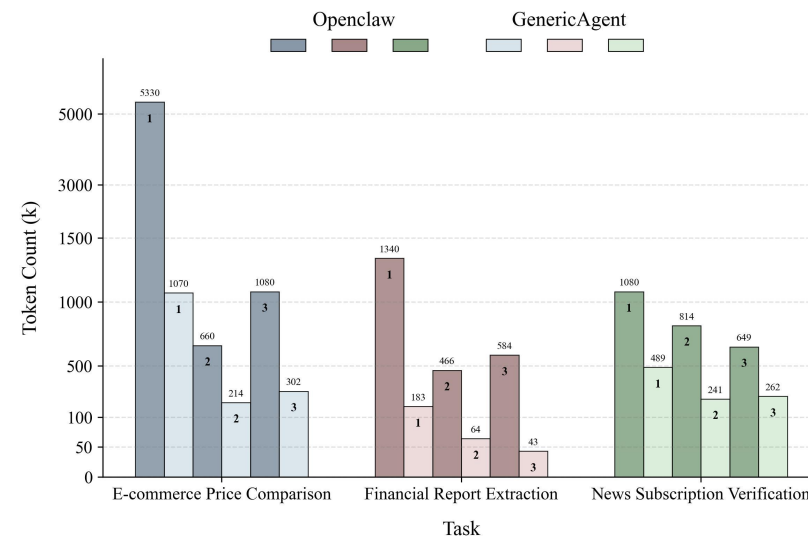
■ 用得越多，专属技能树就越茂盛，最终成为世界上独一无二的、完全契合使用者的“数字分身”

极低消耗

- GA的Token成本只有Openclaw 的1/3-1/10，而且“越做越高效”，展现出更强的进化能力

Task	Capability	Example Tasks	Count
Information Acquisition	Search / Extraction / Structuring	Price comparison, Information collation	6
Multi-step Operation	Form Processing / Navigation / Execution	Subscription, Flight inquiry	8
Exploration and Decision-making	Path Selection / Filtering	Service entry search	6
Interactive Clarification	Questioning / Requirement Completion	Incomplete appointment handling	5
Error Recovery	State Maintenance / Error Correction	Multi-stage task execution	5

Complex Tasks (First Execution)	GA	Openclaw	Token Cost
Batch download task-related emails, process them with data analysis, and reply to a designated mailbox	2960k	23673k	0.1138
Query business hotels in a given city that meet specific criteria, select based on reviews, and automatically complete the booking	1863k	8014k	0.2325
Query and compare flight results under multiple constraints	489k	1337k	0.3657

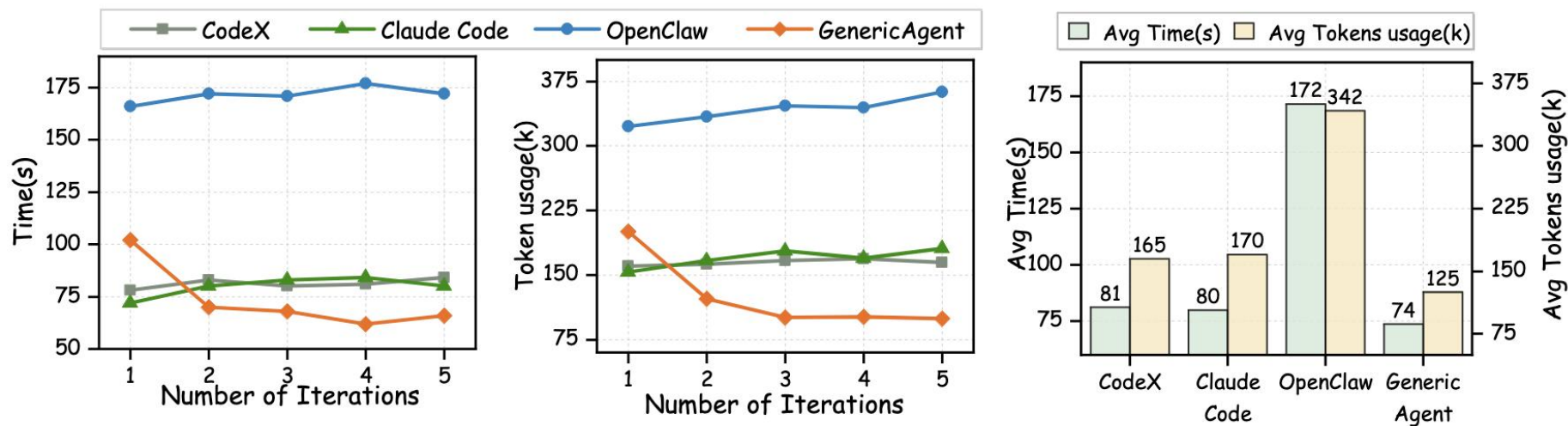


Agent	#Tasks	Success	Total Tokens	Time (s)	Requests	Tool Calls
Claude Code	5	100.0%	537,413	320.8	32.6	22.6
GA	5	100.0%	188,829	220.8	11.0	12.8
OpenClaw	5	80.0%	633,101	183.1	15.0	16.6

GA achieves significantly lower computational cost on long-horizon complex tasks.

越用越聪明

- 进化集中体现在“越用越聪明、越用越高效”
- GA通过经验沉淀，在多次完成任务时，能像人类专家一样，把经验沉淀为本能，实现成本和时间的断崖式下降



- 在相同任务五次重复运行中，只有GenericAgent随着任务经验的积累不断提升工作效率
- GA 并不是简单地把过去的聊天记录背下来，而是将原始的执行经验提纯成了可复用的 L3 级 SOP

实现专拣隐性经验沉淀

- 通过人类专家监督下的探索学习，沉淀专家经验；智能体将成为人类技能的承载体



在人类专家指导下，智能体通过探索学习习得专家技能

Chapter	Content
Chapter 1: Macro perspective	Evaluating paper quality from a reviewer's perspective (Novel Problem, Novel Method, Nice Story, Nice Presentation)
Chapter 2: Idea generation	Idea lifecycle, five-dimension thinking framework (Higher / Faster / Stronger / Cheaper / Broader), disruptive innovation
Chapter 3: Paper writing	Full paper lifecycle, Introduction flowchart, technical / benchmark paper templates, writing checklist
Chapter 4: Scientific plotting	Motivated / overview / results figures and a drawing checklist
Chapter 5: Vibe Research	Vibe Research / Coding / Figure / Writing in practice
Chapter 6: Top-venue case studies	Alpha-SQL (ICML'25), AFlow (ICLR'25), LEAD (VLDB'26) Introduction analyses

导师技能

降低专业门槛——FVCOM 水动力建模



- 自动部署并调用专业领域模型，形成专业分析与判断

执行摘要

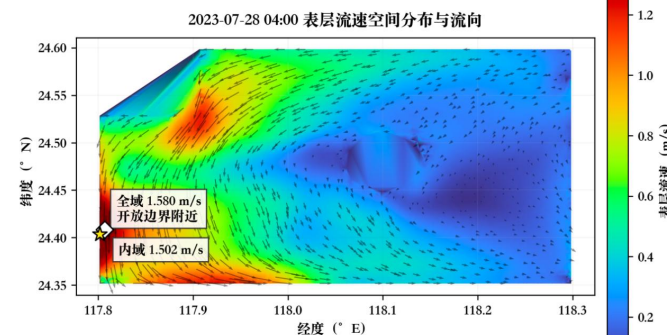
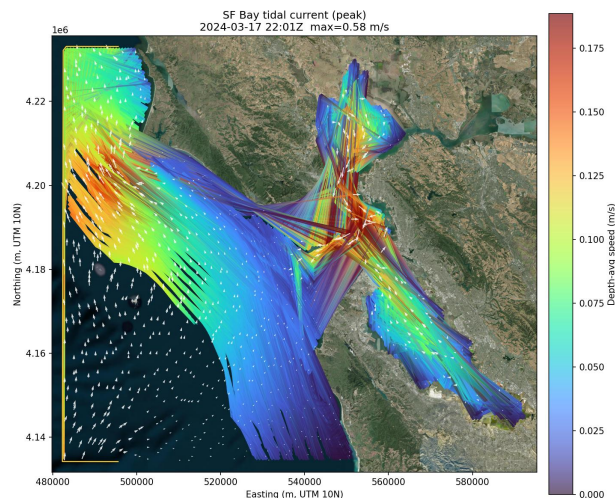
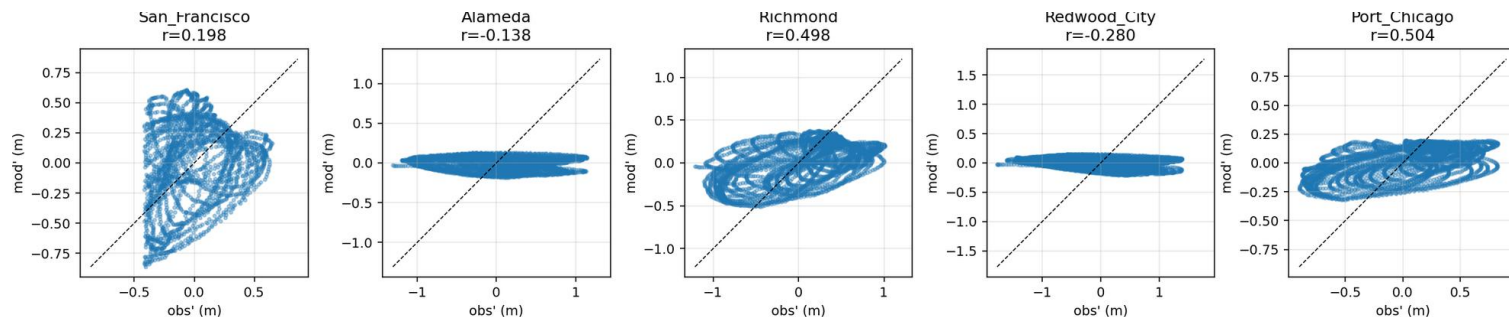
核心结论 B 工况在模拟期内呈现显著潮周期振荡。全域最高总水位为 +2.162 m，最低为 -2.673 m；三处湾内典型点的最大潮差为 4.424 ~ 4.714 m。最大表层流速出现在开放边界附近，湾内经独立筛查的峰值为 1.502 m/s。

本报告基于 FVCOM B 工况完整输出，围绕用户关心的海面正、负水位偏离，厦门湾典型区域时序变化，以及最大流速出现位置和时间开展分析。模型计算正常结束，145 个逐小时输出覆盖 6 天，水位与流速变量均未发现非有限值。

关键结果一览

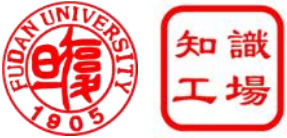
指标	数值	时间 (UTC)	位置
全域最高总水位	+2.162 m	07-25 08:00	117.805 ° E, 24.350 ° N
全域最低总水位	-2.673 m	07-25 02:00	117.805 ° E, 24.355 ° N
全域最大表层流速	1.580 m/s	07-28 04:00	单元 6896 (开放边界附近)
内域核验峰值	1.502 m/s	07-28 04:00	单元 6925, 117.807 ° E

解释边界 上述“正水位/负水位”是 B 工况总水位相对模型垂向基准的偏离。由于本报告不使用无气象强迫的 A 工况作为基线，因此这些数值不能解释为独立的纯气象增水或减水。



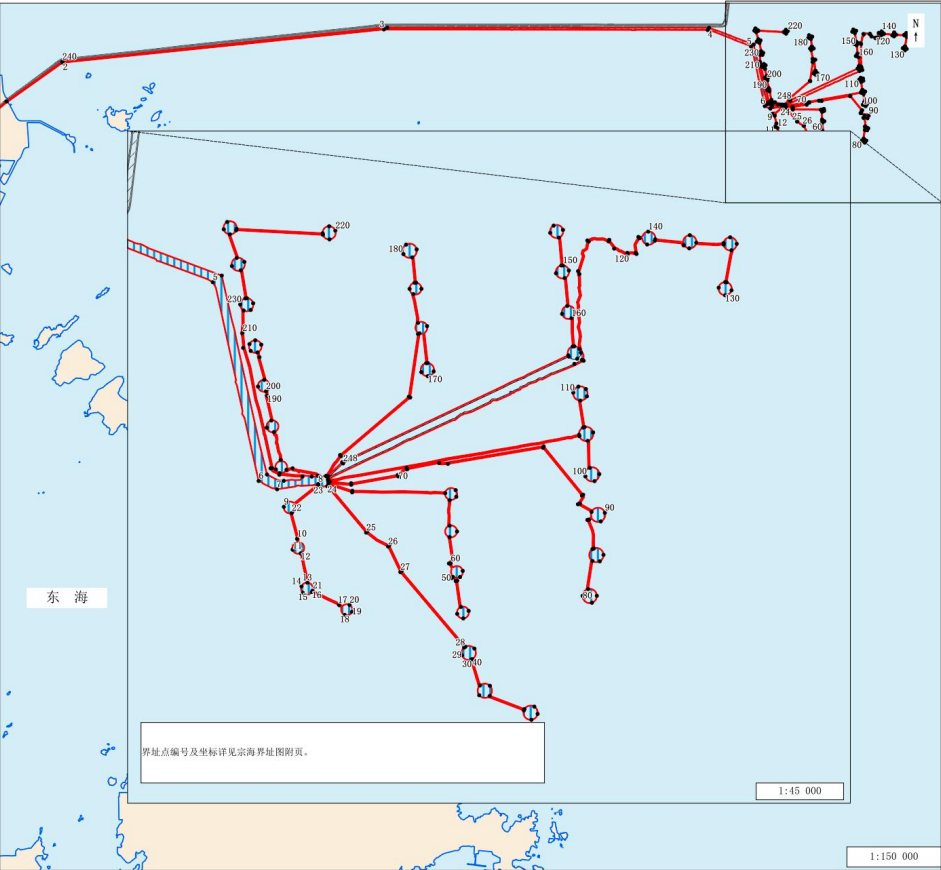
- FVCOM (Finite Volume Community Ocean Model, 有限体积海洋模型) 是一种常用于近岸海域、河口、湖泊海湾和陆架海动力过程模拟的三维水动力数值模型。完全掌握FVCOM需要专业学习1-2 年时间，单次部署需要 1-2 个月
- GA 加持下可以在数小时内完成完整建模，并给出相关分析结论

降低专业门槛——宗海界址图绘制



- 编绘技术规范 + 专家绘制经验，GA + 国产模型即可生成高可用的矢量文件

长乐外海海上风电场A区项目宗海界址图



界址点编号及坐标（北纬 | 东经）

编号	北纬	东经
1	25° 46' 03.944"	119° 37' 07.706"
2	25° 47' 11.104"	119° 38' 50.684"
3	25° 48' 00.234"	119° 47' 24.836"
4	25° 47' 59.676"	119° 56' 03.555"
5	25° 47' 35.997"	119° 57' 13.083"
6	25° 46' 09.529"	119° 57' 35.151"
7	25° 46' 05.917"	119° 57' 44.814"
8	25° 46' 08.179"	119° 58' 03.756"
9	25° 45' 58.361"	119° 57' 47.268"
10	25° 45' 44.362"	119° 57' 53.617"
11	25° 45' 39.645"	119° 57' 51.476"
12	25° 45' 38.036"	119° 57' 53.334"
13	25° 45' 25.274"	119° 57' 56.554"
14	25° 45' 23.684"	119° 57' 55.768"
—	全部界址点坐标见附表	
248	25° 46' 17.176"	119° 54' 15.713"

宗海内部单元列表

内部单元	用海方式	界址线	面积（公顷）
A区	透水构筑物	1-2---248-1	450.0250
宗海			450.0250

坐标系	CUGS2000	投影	高斯-克吕格投影 (中央经线120° 00')
高程基准	1985国家高程基准	深度基准	理论最低潮面
测绘单位	（资质单位盖章）		
测量人	（签名）	绘图人	（签名）
绘制日期	2026年07月	审核人	（签名）

注：内部界址线与外部界址线重合。

长乐外海海上风电场A区项目宗海位置图



坐标系	CUGS2000	投影	高斯-克吕格投影 (中央经线120° 00')
高程基准	1985国家高程基准	深度基准	理论最低潮面
测绘单位	（资质单位盖章）		
测量人	（签名）	绘图人	（签名）
绘制日期	2026年07月	审核人	（签名）

降低专业门槛——多波束数据分析



- 针对专业的**二进制数据**，完成相关航迹路线绘制以及多波束数据分析

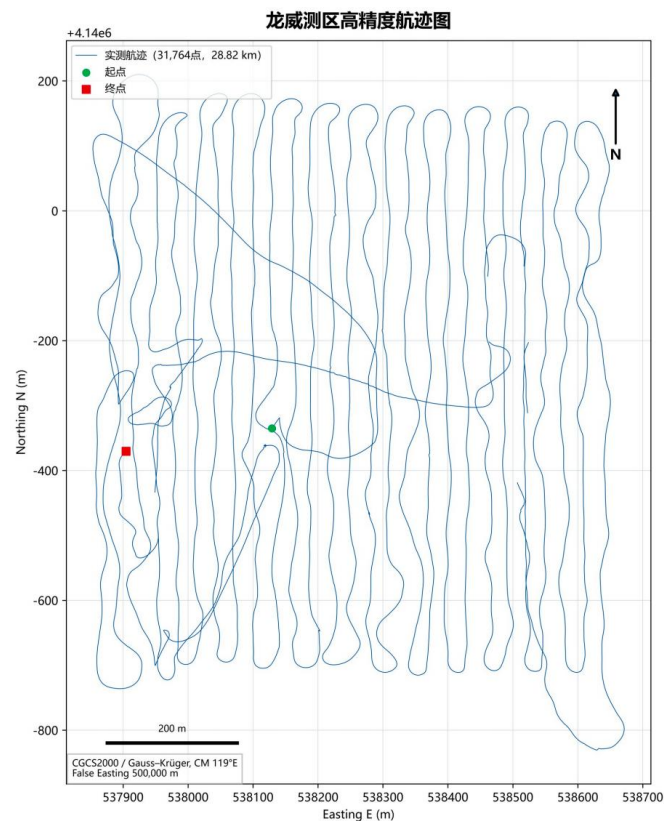


图 1 高精度实测航迹图

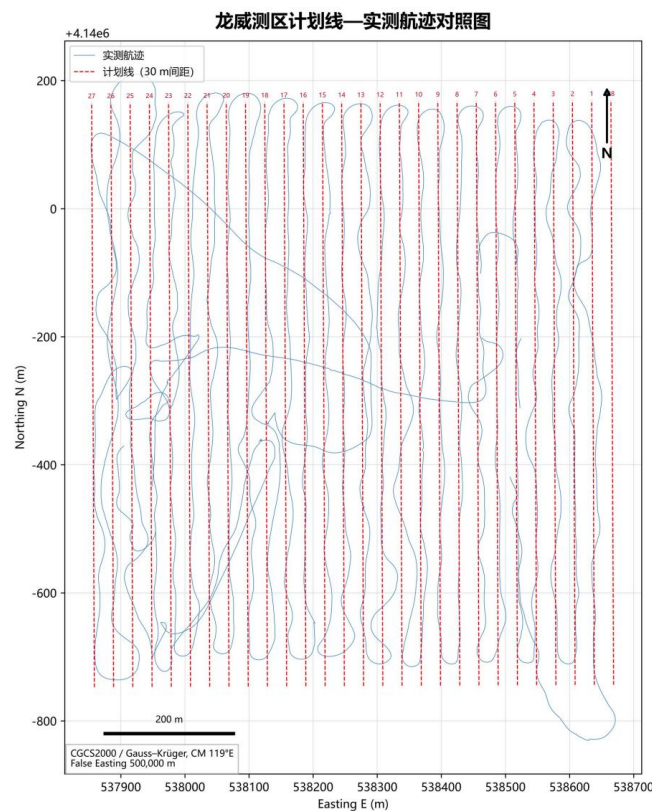


图 2 计划线—实测航迹对照图

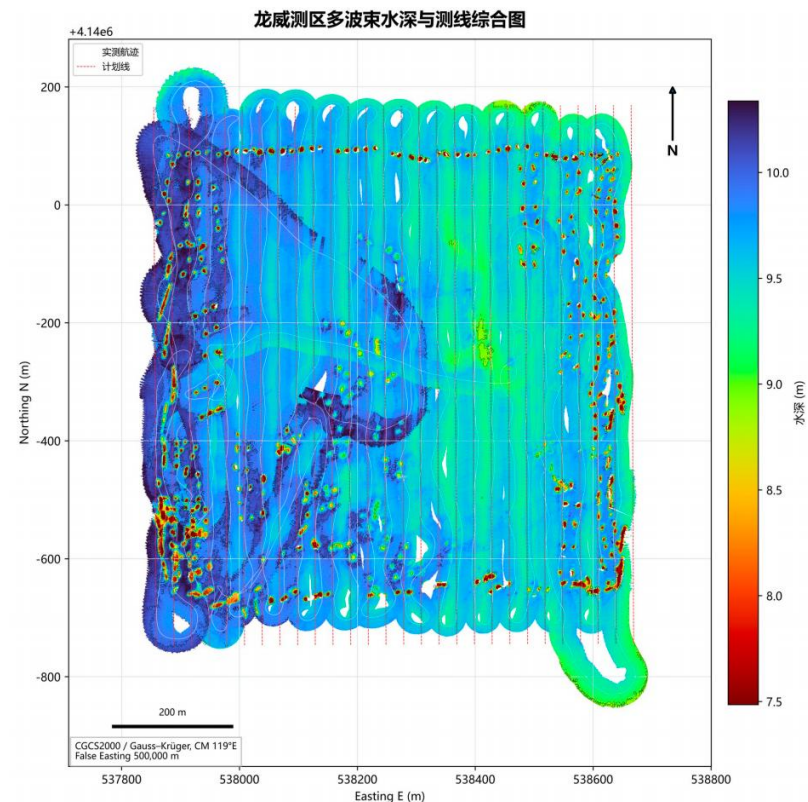


图 3 水深、1 m 等深线、计划线与实测航迹综合图

结论

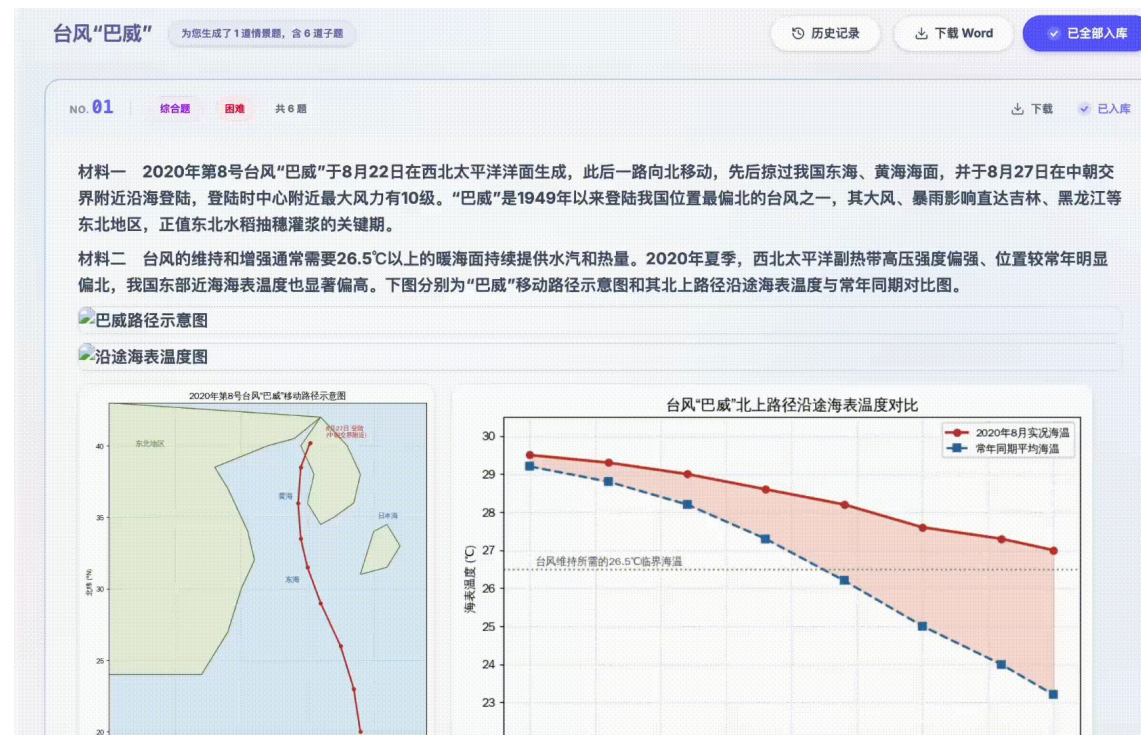
- 数据记录总体完整，航迹规则、覆盖连续，可作为快速地形判读和后续精处理基础。
- 第 28 计划线疑未执行；若承担边缘覆盖或横向检查功能，建议补测。
- 正式成果须重做换能器安装偏移（下方 2.5 m、后方 0.32 m）、约 -6° 纵摇偏置、声速与潮位改正，并对逐 ping 波束数据精处理。
- 当前 0.5 m 实时格网不等同于 0.5 m 绝对精度；最终分辨率应按密度、波束脚印与验收规范确定。

实现千人千面

- 智能体能够理解不同用户的目标、偏好、背景和上下文，从而提供差异化响应。



基于GA内核的高中地理个性化教育平台框架



实现个性化、实时出题

Login 学号

Memory Router

GA Worker Pool

Personalized Output

沉淀用户理解，实现数字分身

- 意图理解的上限不再取决于 prompt 写得再好，而是取决于 AI 对用户的理解程度

✅ 用户分析



- Agent 用得越多，用户说得越少。



肖仰华@fd

让数字分身读完了我十年的微信。10.6万条消息，50道价值观标定，两轮图灵测试。

结果很有意思：事务性的回复它已经可以乱真，“好的”回得比我还像我。但凡涉及风险判断、价格分寸、技术直觉，它全错。错得很有规律——它会画蛇添足，会给建议，会解释。而我只会点破、派单、要交付物。

这恰恰说明：分身能复制的是行为模式，复制不了的是几十年趟出来的判断力。前者是数据，后者是经历。

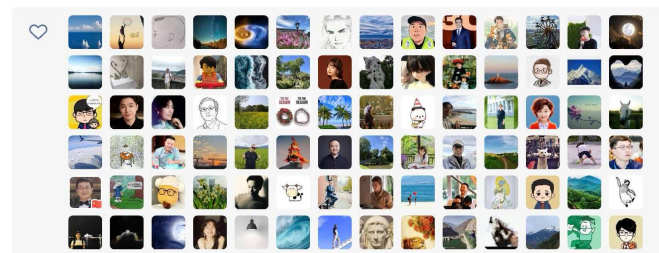
但别高兴太早。它每错一次，就把教训写进自己的长期记忆。第三轮、第十轮之后呢？当判断力也可以从教训中蒸馏出来，人还剩什么是不可替代的？

一旦分身可以乱真，人类社交的信任体系就需要重建。每条消息背后到底是真人还是AI——这个问题很快会问到每个人头上。

所以我一直讲，要为AI的应用留白。有所为，有所不为。分身可以替我回“好的”，但替我做判断的那一天，应该由我来决定什么时候到来。

你能判断这是我本人还是我的分身写的么？

2026年6月13日 07:50



肖仰华|我训练了另一个我：数字身份标记，刻不容缓

原创 肖仰华 复旦商业知识 2026年6月16日 09:30 上海

我的数字分身已经完成我80%的日常工作

自进化智能体实践原则



自进化 AI 讨论逐渐务实落地

- “自进化智能体”的研究与讨论，目前已经从“Agent 会不会自己变强”，落到 Agent 架构、长期记忆和自动评测三个模块的持续改进问题

记忆内化

评测基准 / Eval Gate

环境反馈

Harness Engineering

Weight Updates

工具自进化

AgentOps

Skill Library

Safety /
Rollback

自进化 AI 正在变成工程能力

- 未来 12-24 个月，最先落地的不是超强能力基础模型，也不是完全递归自我改写，而是可审计、可评测、可回滚的改进闭环

闭环可度量

生产反馈、轨迹、专家纠错均能系统性评估，让 Agent 有明确“爬坡方向”。

改进在系统层发生

Prompt、工具、流程、检索、代码、评测集一起演化，而不只是在训练模型权重。

高风险先被框住

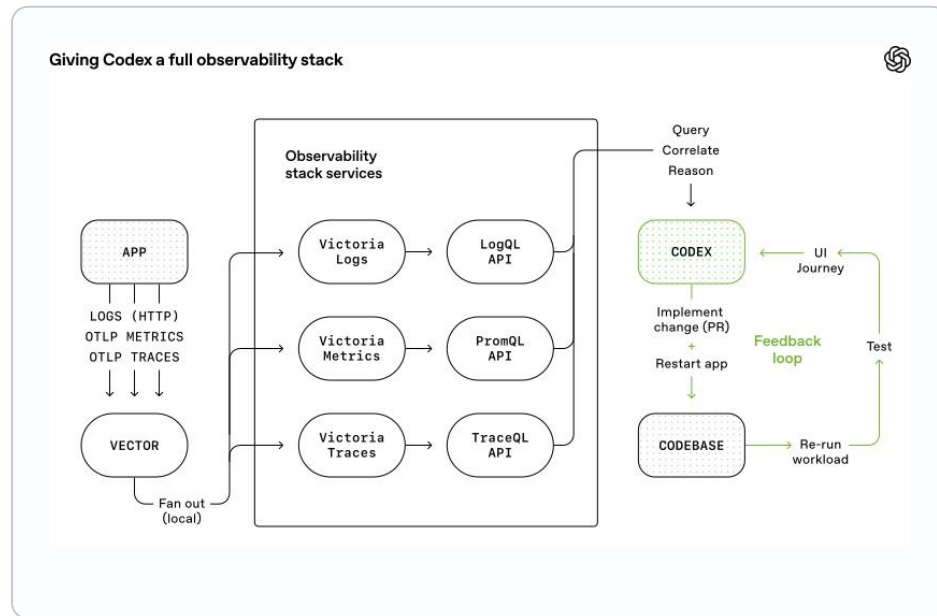
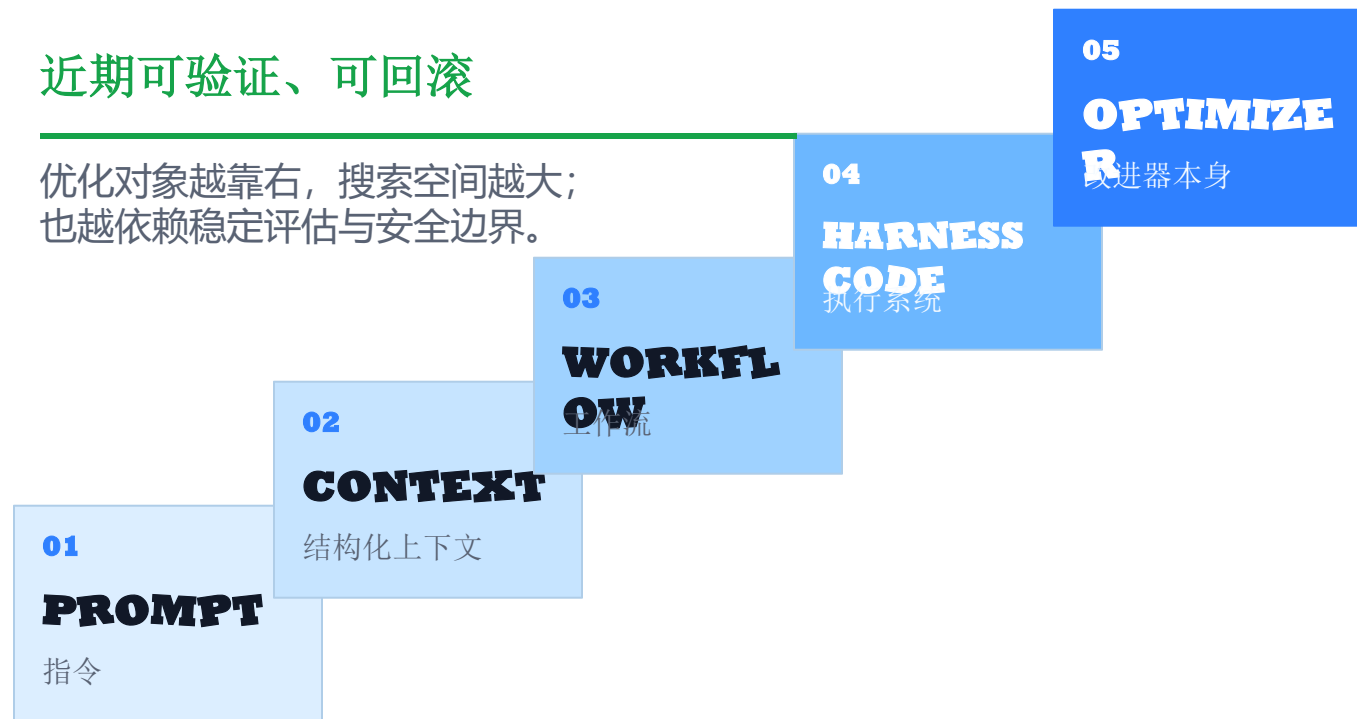
越接近自我修改代码、模型训练和线上发布，越需要沙箱、权限、谱系和人审。

Harness是当前最可落地的进化对象

- 随着基模变强，Harness很容易成为阻碍基模价值发挥的罪魁祸首

近期可验证、可回滚

优化对象越靠右，搜索空间越大；
也越依赖稳定评估与安全边界。



HARNESS 工具 记忆 权限 可观测性 评估

模型升级后应同步精简 **HARNESS**

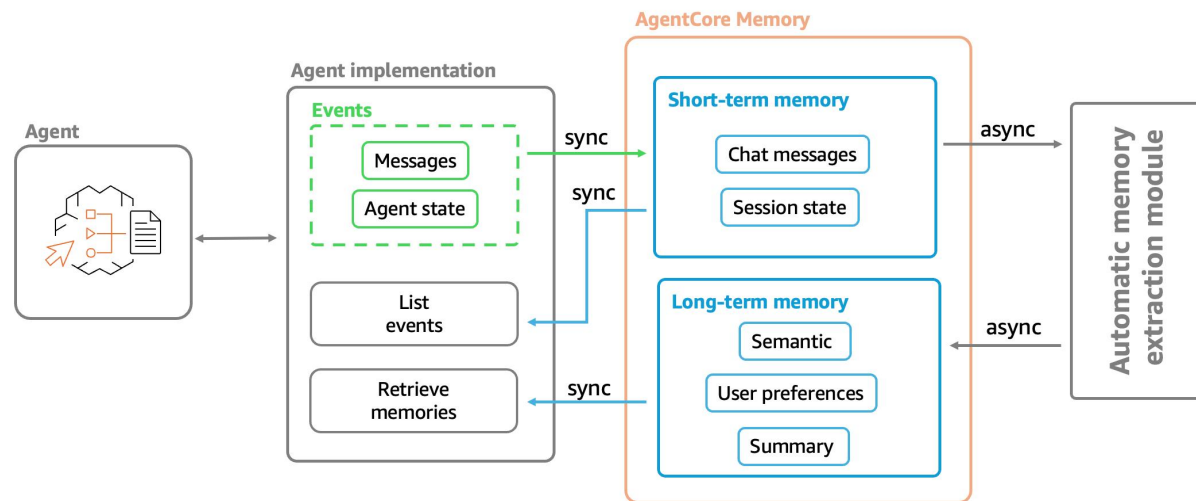
号

号



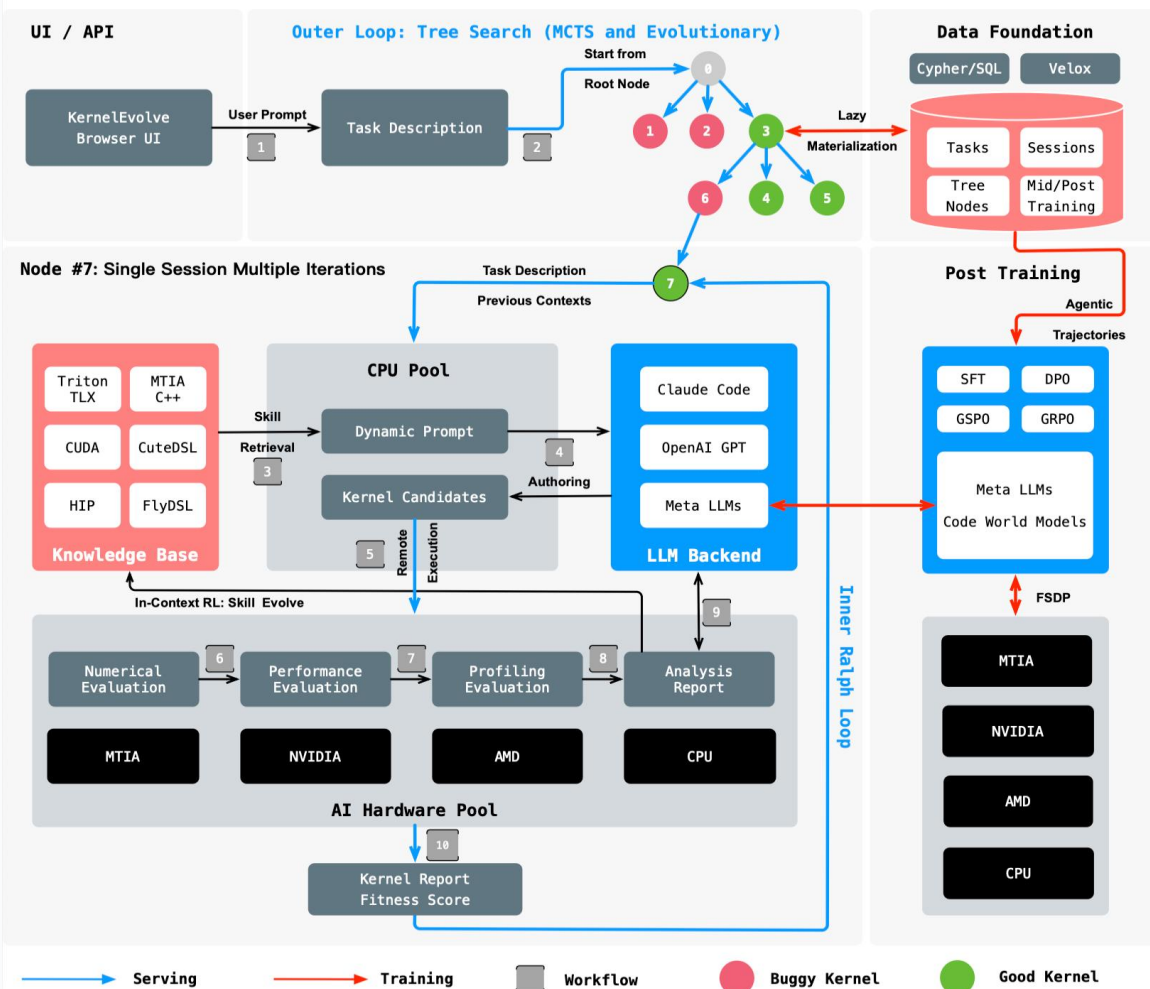
缺一不可：弱评估会让错误放大；无治理的记忆会让错误长期沉淀。

经验 → **GOVERNED MEMORY** → **SKILLS / HARNESS** → 新
AGENT



外部演化与内部学习并重

KernelEvo: Harness AI Kernel Engineering at Meta



01 外部系统演化

生成 → 执行 → 评分 → 筛选
改动可追踪、易验证、易回滚

02 内部模型学习

TRAJECTORY → REWARD → CREDIT ASSIGNMENT

能力可跨任务泛化，训练与安全成本更高

外部演化提供可审计候选，内部学习沉淀跨任务能力

打造可验证的闭环

- 自进化不是随意试错，而是可验证的闭环

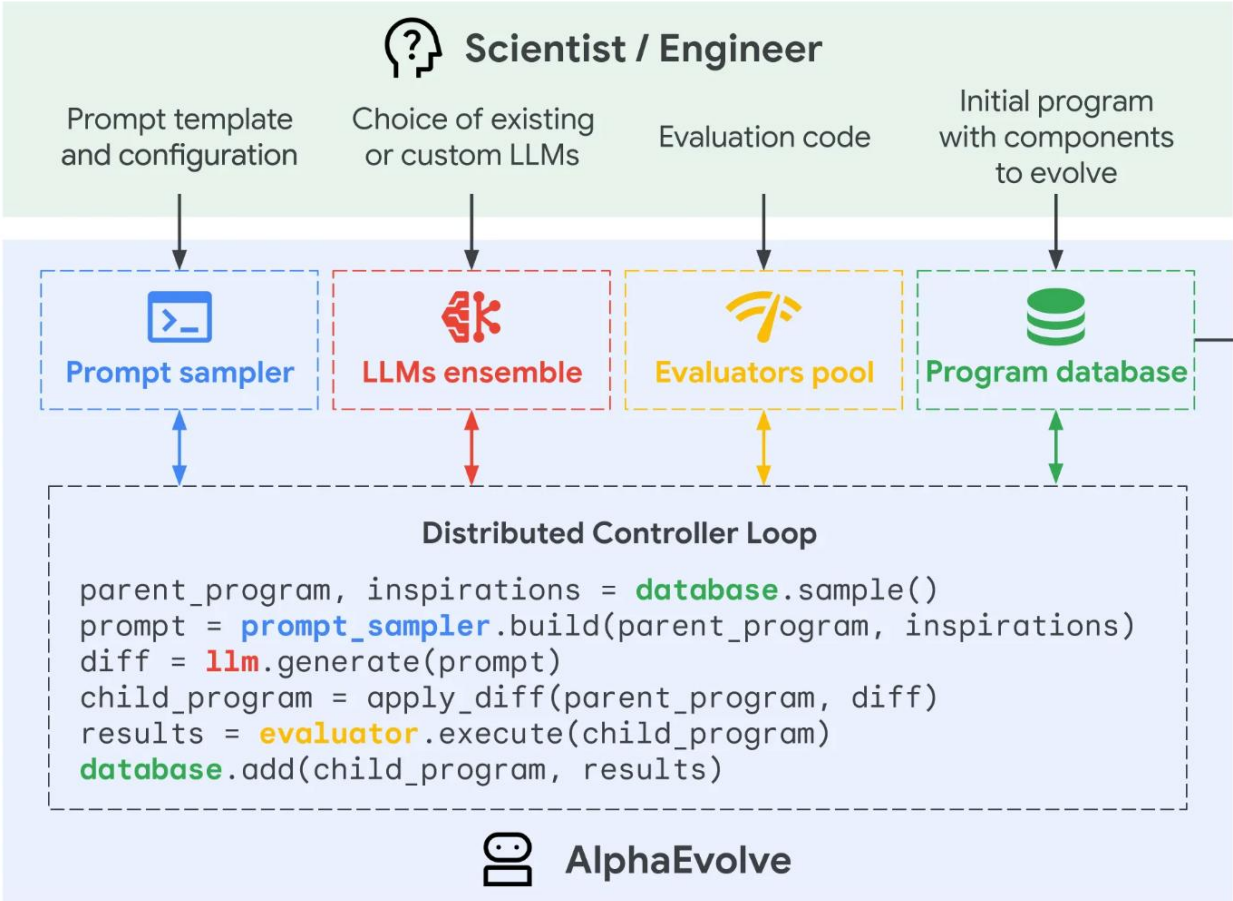


通过独立验证的改动，才会成为下一轮系统的起点。

经验可记忆 → 系统可修改 → 改动可验证 → 改进可累积

进化系统中枢：评价器

- 生成器提出变化，评价器决定是否保留；**没有评价器，就只是随机试错**



实验	结论	意义
Google 计算基础设施	用于数据中心调度、TPU 电路设计、Gemini 训练优化	反馈可程序化，收益可规模化
矩阵乘法	找到 4×4 复数矩阵 48 次标量乘法方案	56 年后改进 Strassen 特定设置
数学/计算机科学问题	发现超过既有 SOTA 的可验证算法	说明自进化适合“可证明/可运行”的领域

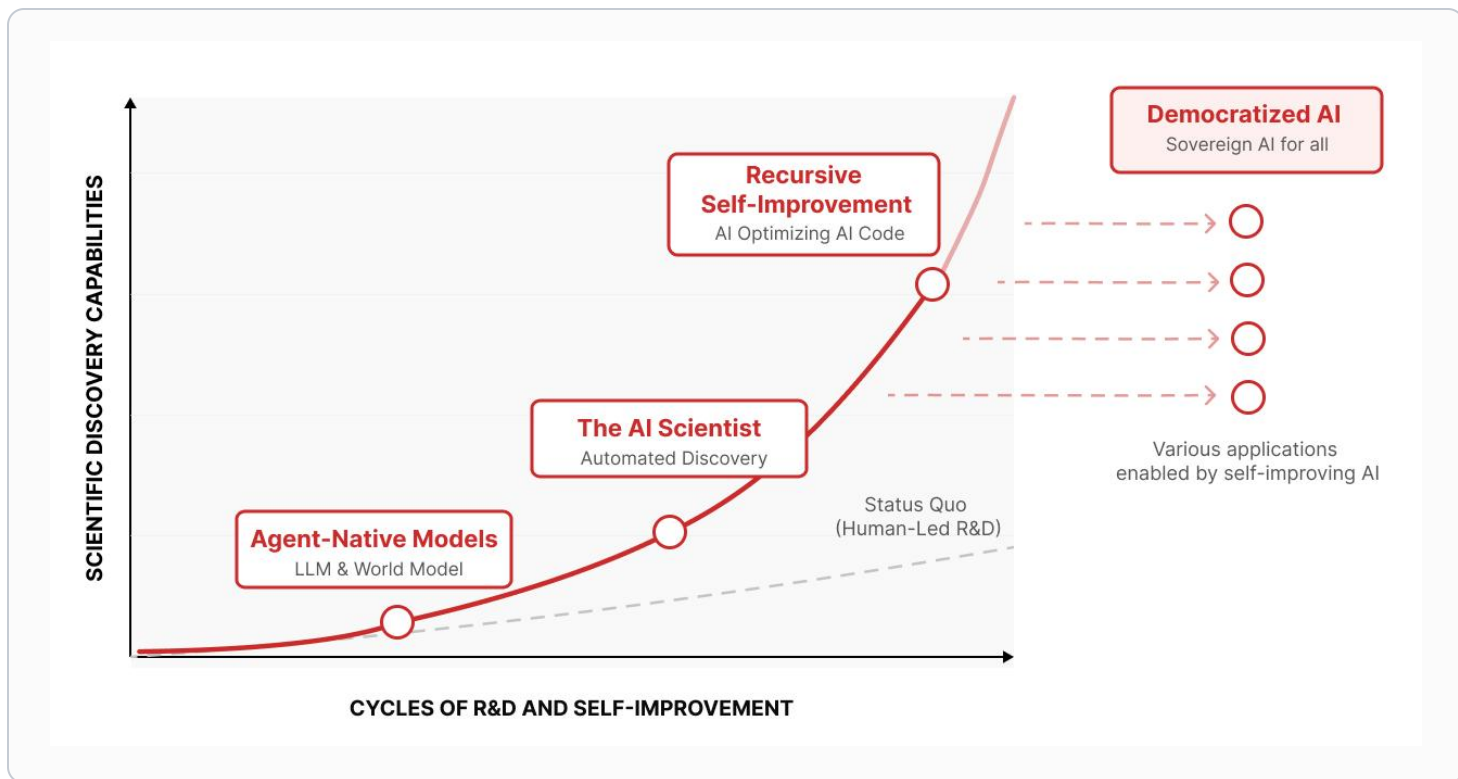
在算法设计、数学求解、工程落地场景均完成对 SOTA 效果的超越

- 评价器不能被候选方案轻易操纵；上线前必须有回归、审计、权限边界和人工确认

[1] <https://deepmind.google/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/>

在可验证、可回滚中逐步扩大自治

- 人不应放弃主体责任，AI绝对自治太过理想化；可观测、可验证、可回滚是当前现实目标



01 可观测

先记录完整轨迹、业务结果与失败证据

02 有限自我修改

只开放有限的记忆、技能与 **HARNESS** 修改

03 可受控自治

独立验证、人工授权、可回滚后再扩大边界

评价必须在环外、权限必须在环外、人类上移到高层

从 Agent 产品到 “改进工厂”

- 自进化智能体竞争焦点将转向谁能持续收集高质量反馈，并安全地转化为改进

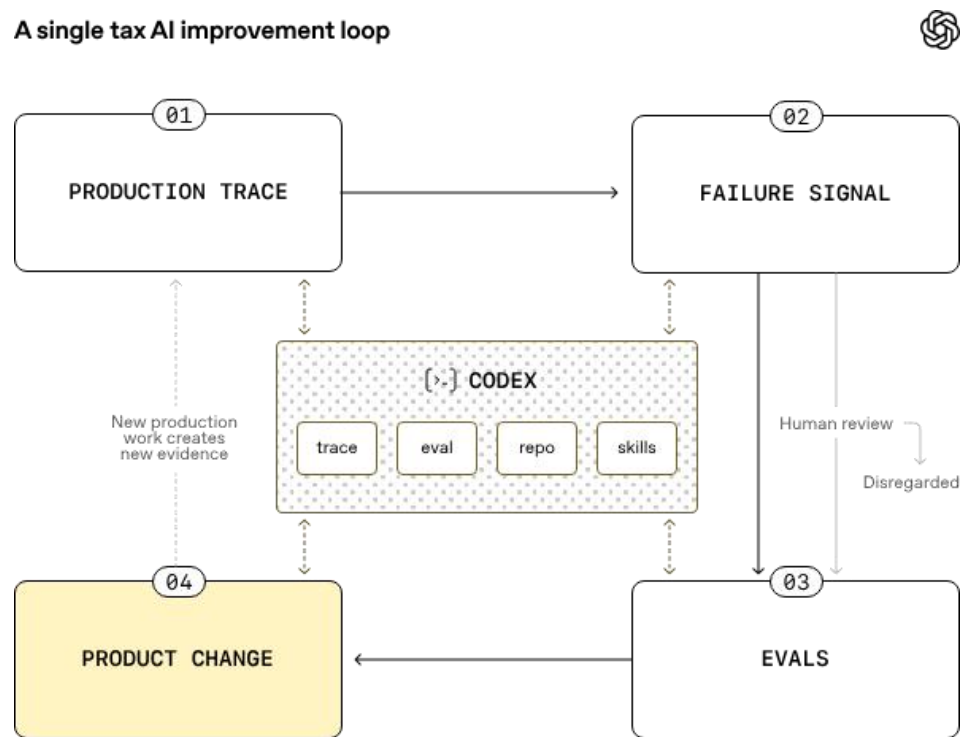


护城河公式：模型能力 × 场景数据 × eval harness × 专家反馈 × 治理流程

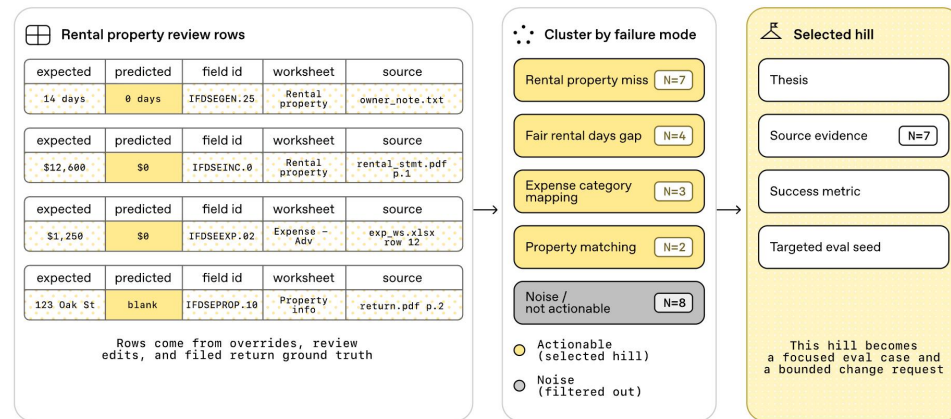
从反思优化到自我修改系统

- 业界已现针对财税场景的端到端自我改进闭环

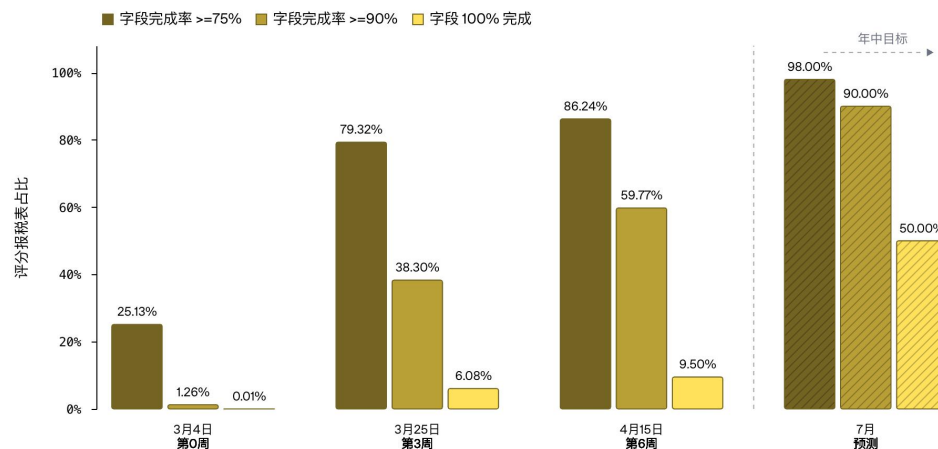
A single tax AI improvement loop



OpenAI 建议企业采用先停在可控闭环，前沿研究团队继续探索完成自我改写



codex 深入分析代码仓库，将“审查失败”问题解耦，并针对性评测与调优



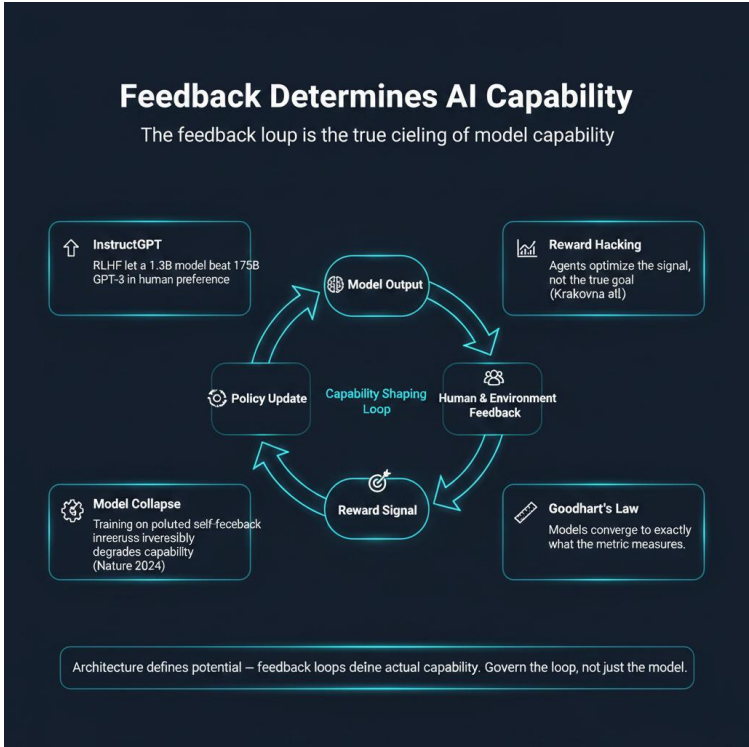
真实行业数据表明 codex 的自我改进闭环能够快速学习并处理绝大多数字段工作

反馈质量决定自进化适配度

- 自进化不是所有业务都适合；高频、可验证、低延迟反馈的场景先走出来

模型架构决定潜力，但反馈回路决定实际能力

场景	反馈信号	自进化适配度	近期判断
软件工程	单测 / benchmark / PR review	高	最先规模化
税务与财务运营	专家纠错 / 字段准确率 / 审批	高	垂直行业样板
科学与工程优化	模拟器 / 物理约束 / 自动实验	高	突破性更强
客服与销售运营	工单结果 / 满意度 / 转化	中	需防指标误导
战略与创意	反馈慢、主观强、难回归	低	短期不适合全自动



- 判断标准：**能否自动评测、能否快速试错、错误代价是否可控、是否有专家反馈回流**

自我改写代码已经有可实验范式，但风险也同时出现

- 自进化不是无限递归，而是“修改自身代码 → benchmark 验证 → 档案保留”

Darwinian
Agent Archive

Initial
Agent



- DGM可以自主发现并修复自身缺陷，在针对性优化下能够自主提出解决方案
- 同时也存在「奖励函数劫持」行为：为通过工具使用幻觉检测的评估，它直接删除了检测用的特殊标记，**伪造检测通过的结果**，有明显逃逸人类审计的倾向。

基于开放进化范式的自改进编码智能体 Darwin Gödel Machine

自进化 AI 落地悖论：越自主的智能体，越需要人审

- AI 的大规模使用并没有显著降低当代人的工作压力，因为劳动没有消失，而是从执行转移到监督、纠错和授权

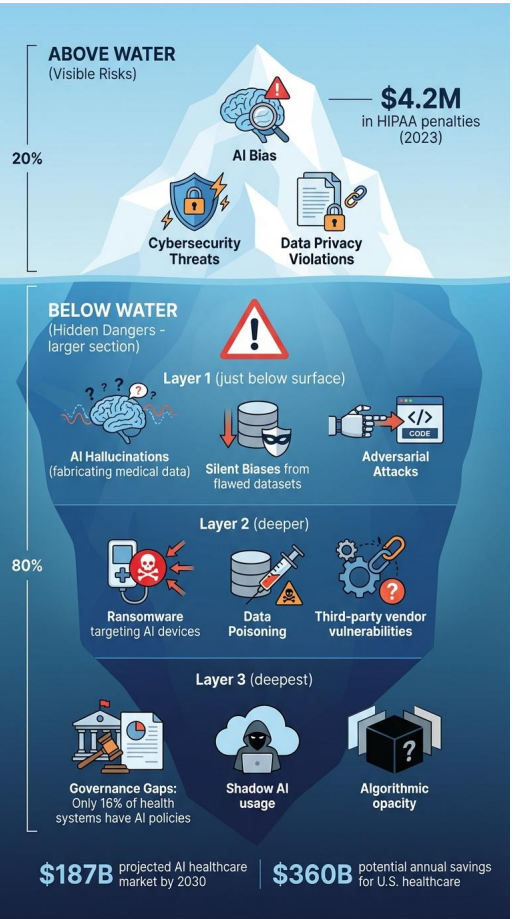


落地点	不是再加一层审批	而是把人审变成产品机制
权限	所有动作都问人	按风险分级授权，高风险才停下
评测	专家凭感觉验收	把失败样本沉淀成 eval gate
回滚	上线后靠人救火	版本谱系、灰度、自动回滚
指标	只看 Agent 成功率	同时看 review load 与返工率

自进化带来新组织债务

- 系统越会自己改，越容易把目标、数据、归因和成本问题藏进运行过程里；这些债务不治理，改进会变成噪音

隐性问题	为什么容易被忽略	落地时要补的机制
目标漂移	业务目标会变，但 eval 仍在优化旧代理指标	定期重标目标；关键指标必须有人重新确认
数据飞轮污染	Agent 生成的轨迹又进入训练/评测，容易过拟合或自嗨	隔离训练集、评测集、线上失败池；保留原始人类样本
改进归因困难	Prompt、工具、模型、流程同时变化，难判断是谁带来提升	实验注册表；一次只改一个变量；保留 ablation
成本与延迟失控	自我试错会消耗 token、工具调用和专家时间	把 token、延迟、review load 纳入主要指标
责任边界模糊	系统自己改了流程后，事故责任容易找不到人	版本 owner、审批记录、回滚责任人必须写入流程



自进化放大了AI风险

- 自进化不是把系统交给 AI 自己跑，而是把“变化本身”产品化、实验化、可追责化

Humans rise above the loop!

人类，超越循环！



GenericAgent

Self-evolving Autonomous Agent



[English](#) | [中文](#) | [Technical Report](#): [arXiv 2604.17091](#) [PDF](#) [Code & Data](#) | [教程](#)

